



Web Services Internationalization Usage Scenarios

W3C Working Group Note 30 July 2004

This version:

<http://www.w3.org/TR/2004/NOTE-ws-i18n-scenarios-20040730>

Latest version:

<http://www.w3.org/TR/ws-i18n-scenarios>

Previous version:

<http://www.w3.org/TR/2004/WD-ws-i18n-scenarios-20040512>

Editors:

Debasish Banerjee, IBM

Martin J. Dürst, W3C [<duerst@w3.org>](mailto:duerst@w3.org)

Mike McKenna, University of California

Addison Phillips, webMethods [<aphillips@webmethods.com>](mailto:aphillips@webmethods.com)

Takao Suzuki, Microsoft [<takaos@microsoft.com>](mailto:takaos@microsoft.com)

Tex Texin, XenCraft

Mary Trumble, IBM

Andrea Vine, Sun Microsystems

Kentaroh Noji, IBM

This document is also available in these non-normative formats: [XML](#).

[Copyright](#) © 2004 [W3C](#)® ([MIT](#), [ERCIM](#), [Keio](#)), All Rights Reserved. W3C [liability](#), [trademark](#), [document use](#) and [software licensing](#) rules apply.

Abstract

This document describes internationalization usage patterns and scenarios for Web services. It also provides guidance for implementers of Web service technologies, suggesting methods for dealing with general international interoperability issues in services and service descriptions. One goal of this document is to provide a template for Web service designers to implement international capabilities in their services.

Status of this Document

This section describes the status of this document at the time of its publication. Other documents may supersede this document. A list of current W3C publications and the latest revision of this technical report can be found in the

[W3C technical reports index](http://www.w3.org/TR/) at <http://www.w3.org/TR/>.

This document is a [W3C Working Group Note](#), made available by the of the [W3C Internationalization Working Group](#) ([Web Services Internationalization Task Force](#)) as part of the [W3C Internationalization Activity](#). It describes internationalization usage scenarios for Web services and is intended for review by W3C Members and other interested parties. It is also intended to serve as a basis for future work on Web service internationalization.

The Internationalization Working Group (Web Services Internationalization Task Force) thinks that this document has reached a sufficient level of maturity to be published as a Working Group Note, and does not intend to issue new versions in the near future. This does not exclude that the document may be updated at a later stage, after more experience has been gained with the internationalization of Web services.

The Internationalization Working Group or its successor will keep track of any further comments and discussion relating to this document, and invites any such comments or discussion. Discussion of this document takes place on the public mailing list public-i18n-ws@w3.org. To contribute, please subscribe by sending mail to public-i18n-ws-request@w3.org with subscribe as the subject. The [archive](#) of this list can be read by the general public. Please send comments on this document to the www-i18n-comments@w3.org mailing list ([public archive](#)). Please use [Web Services] or [WSUS] in the subject.

Publication as a Working Group Note does not imply endorsement by the W3C, including the Team and Membership.

Table of Contents

1

[Introduction](#)

[1.1 Audience for This Document](#)

[1.2 Scope](#)

2 [Introduction to Web Services](#)

[2.1 Basic Framework: Anatomy of a Web Service Interaction](#)

[2.1.1 Discovery](#)

[2.1.2 Request](#)

[2.1.3 Response](#)

3 [Introduction to Internationalization: Definitions for a Discussion of Web Services](#)

[3.1 What are Internationalization and Localization?](#)

[3.1.1 Relationship of Locale to Natural Language](#)

[3.1.2 I-025: Specifying and Exchanging International Preferences in Web](#)

[Services](#)

[3.1.3 Locales in Web Service Descriptions](#)

[3.1.4 Locales in SOAP](#)

- 3.1.5 [Faults, Errors, and Human Readable Text](#)
- 3.2 [Locale Independent vs. Locale Dependent Data](#)
 - 3.2.1 [Textual vs. Binary Representations](#)
 - 3.2.2 [Locale-Dependent XML Schema Datatypes](#)
 - 3.2.3 [Examples](#)
- 4 [Basic Web Service Internationalization Scenarios](#)
 - 4.1 [Locale Patterns in Web Services](#)
 - 4.1.1 [The Travel Application](#)
 - 4.1.2 [Locale Neutral](#)
 - 4.1.2.1 [Example: 'GetArrivalTime' Returns Flight Arrival Time](#)
 - 4.1.3 [Client Influenced](#)
 - 4.1.3.1 [Example: 'getItinerary' Get Flight Information in the Requester's Language](#)
 - 4.1.3.2 [Service Description](#)
 - 4.1.4 [Service Determined](#)
 - 4.1.4.1 [Example: 'flightCheck' Service](#)
 - 4.1.5 [Data Driven](#)
 - 4.1.5.1 [Example: 'getWeightRestrictions' Gets Flight Luggage Restrictions](#)
 - 4.1.5.2 [Example: Stored User Preferences](#)
 - 4.1.5.3 [Example: Data from Service to Service](#)
 - 4.2 [Locale and Language Dependency in Message Exchange Patterns](#)
 - 4.2.1 [I-009: One Way Messages](#)
 - 4.2.2 [I-018: Data Associated with a Default Attribute](#)
 - 4.2.3 [I-013: Conflicts Between Requester's Expectations and Service's Locale](#)
 - 4.3 [Fault Handling](#)
 - 4.3.1 [I-004: Producing Fault Reasons in All Available Languages](#)
 - 4.3.2 [I-005: Language Matching for Fault Reason Messages](#)
 - 4.3.3 [I-008: Locale Sensitive Formatted Data in SOAP Fault Messages](#)
 - 4.4 [Legacy Issues](#)
 - 4.4.1 [Pandora's box: Using Non-internationalized Data Structures](#)
 - 4.4.2 [I-019: Locale Dependent Datatypes](#)
 - 4.4.3 [Existing Web Services](#)
 - 4.5 [Character Encodings and Web Services](#)
 - 4.5.1 [SOAP Documents and the MIME Charset Parameter](#)
 - 4.5.2 [Character Encoding of Attachments](#)
 - 4.5.3 [Unsupported Charset in Request Scenario](#)
 - 4.5.4 [Unsupported Charset in Response Scenario](#)
 - 4.5.5 [Unsupported Characters](#)
 - 4.5.6 [Legacy Application Use of Non-Unicode Character Encodings](#)
 - 4.5.6.1 [Calling the Service Requires Transcoding](#)
 - 4.5.6.2 [Service's Internal Implementation Performs Transcoding](#)
 - 4.5.7 [Variability in Transcoding Scenario](#)
 - 4.6 [Passing or Matching International Preferences](#)
 - 4.7 [Intermediaries and Internationalization](#)
 - 4.7.1 [I-020: Correlation of Data Between Services in Different Languages](#)
 - 4.7.2 [I-007: Interaction of Optional Locale and Language Negotiation and Chained Services](#)
 - 4.7.3 [I-012: Caching](#)
 - 4.7.4 [Caching with Locale Information in SOAP Headers](#)
 - 4.8 [SOAP Header Structures](#)
 - 4.8.1 [Character Encoding Conversion Scenario](#)

- 4.9 [Service Discovery](#)
 - 4.9.1 [Searching for Web Services Using UDDI](#)
 - 4.9.2 [I-026 Searching for Service Descriptions Using My Language](#)
 - 4.9.3 [I-027: Searching for Services Specific to a Region](#)
- 4.10 [Introspection of Services When Generating WSDL](#)
- 4.11 [Ordering, Grouping, and Collation](#)
- 4.12 [Natural Language Text Search](#)
 - 4.12.1 [Language-Neutral Natural Language Text Search](#)
 - 4.12.1.1 [Unicode Normalization](#)
 - 4.12.1.2 [Catalog or Index in Multiple Languages](#)
 - 4.12.2 [Language-Specific Natural Language Text Search](#)
 - 4.12.2.1 [Keyword Searching](#)
 - 4.12.2.2 [Gender and plural variants](#)
 - 4.12.2.3 [Orthographic Variation in Searching \('like' clauses\)](#)
 - 4.12.2.4 [Use of Intermediary Translation and Dictionary Look-Up Service](#)
 - 4.12.2.5 [Phonetic Searches](#)
- 4.13 [Locale Sensitive Presentation and Human Readable Text](#)
 - 4.13.1 [I-021: Data Formatting for End User on Receiver Side](#)
 - 4.13.2 [I-022: Data Formatting on Sender Side](#)
 - 4.13.3 [Enumerated Values and Object Names](#)
 - 4.13.3.1 [Use of Default English-like Names](#)
 - 4.13.3.2 [Types of Names](#)
- 4.14 [Data Structures](#)
 - 4.14.1 [Times and Time Zones](#)
 - 4.14.2 [Calendars and Dates](#)
- 4.15 [Legal and Regulatory Goobers](#)
 - 4.15.1 [Modeling Tax, Customs, Legal, and Other Cross-Border and Cultural Considerations](#)
- 4.16 [Transports](#)
 - 4.16.1 [HTTP](#)
 - 4.16.2 [FTP](#)
 - 4.16.3 [SMTP](#)
 - 4.16.3.1 [MIME Tags](#)
 - 4.16.4 [IRIs, URIs, and fun stuff](#)
- 4.17 [Orchestration and Choreography](#)

Appendices

A

[References](#) (Non-Normative)

B [Acknowledgements](#) (Non-Normative)

C [Heisei](#) (Non-Normative)

1 Introduction

This document describes a variety of Web services internationalization usage scenarios and use cases.

The goal of the

[Internationalization Web Services Task Force](#) is to ensure that Web services have robust support for global use, including all of the world's languages and cultures.

The goal of this document is to examine the different ways that language, culture, and related issues interact with Web services architecture and technology. Ultimately this will allow us to develop standards and best practices for implementing internationalized Web services. We may also discover latent international considerations in the various Web services standards and propose solutions to the responsible groups working in these areas.

1.1 Audience for This Document

This document addresses several audiences.

One audience consists of developers of Web services, including the service itself, as well as the Web service description, message exchange pattern, data structures, and discovery. Web service developers need to design services with language or cultural requirements in mind and provide for these options in the data structures and message exchange patterns used by their services.

Another audience consists of developers of Web service technologies, such as Web service servers ("providers"), directory services, SOAP clients, and the like. These technologies need to provide support that enables Web service developers to create, deploy, and manage internationalized services. This includes managing multilingual configurations, performing language negotiation, providing suitable infrastructure for obtaining and managing international preferences (such as language or locale), and mapping or transforming data appropriately.

1.2 Scope

The usage scenarios in this document are generally limited to the following W3C technologies and deliverables:

- Web Services Architecture Documents

[\[WSA\]](#) [\[WSAUS\]](#) [\[WSAR\]](#) [\[WSAG\]](#)

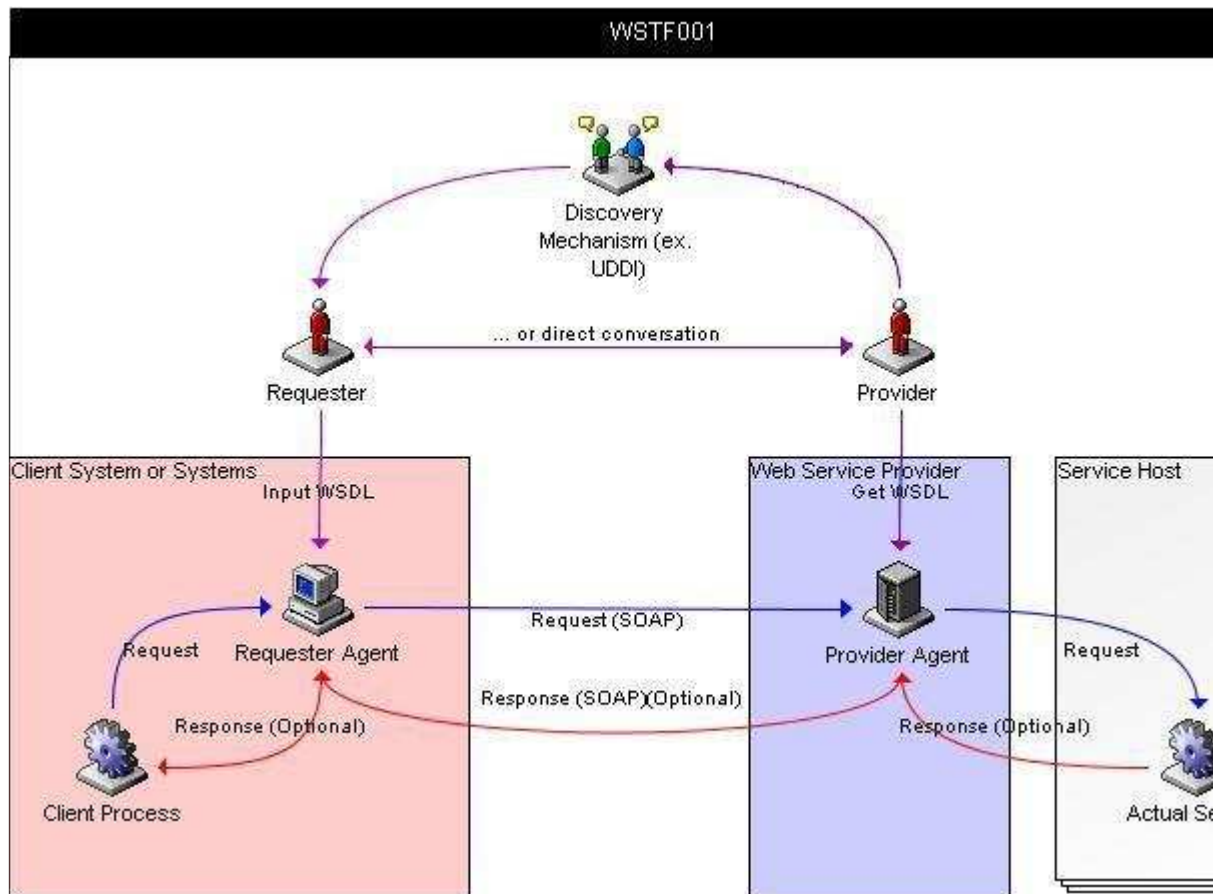
- SOAP V1.2 Documents [\[SOAP-0\]](#) [\[SOAP-1\]](#) [\[SOAP-2\]](#) [\[SOAP-AF\]](#) [\[SOAP-EB\]](#)

- WSDL V1.2 Documents [\[WSDL-V20\]](#) [\[WSDL-B\]](#)

2 Introduction to Web Services

This section describes the basic infrastructure of Web services necessary to understand this document. Some definitions of common Web services technology appear in Appendix D.

2.1 Basic Framework: Anatomy of a Web Service Interaction



There are three basic parts to a Web services interaction. The first part is discovery and configuration. The second part is the request. The third part is the optional response. In the diagram above, the purple arrows are the discovery, the blue arrows are the request, and the red arrows are the response.

It is important to distinguish between the actual *service* and the Web service *provider* or *agent*. The service is the function, method, or other logic that actually is "the service". The provider is the process that receives and emits SOAP messages. In the diagram above, we show the client process and the requester agent as being in a single machine and process, while the provider agent and the actual service are in separate processes. Neither of these is necessarily the case: the provider agent may host the service inside its process, just as the client process and requester agent might be in separate processes or on separate machines.

2.1.1 Discovery

Discovery of Web services can be done in many ways. For example, the system administrators on either end of the transaction may just discuss their objectives or exchange Web service description files (WSDL). Other options include a UDDI registry or other automated mechanism.

The Web service description forms the service contract for the service. It contains the information about how to invoke the service and what, if any, response to expect. This information is called the *message exchange pattern*. The message exchange pattern includes the data structures passed to the service itself and the data structures returned (if any). It may also describe *headers* that contain the contextual information, either required or optional, that the service may need for features such as quality-of-service, security, transactionality, and so forth.

Each service or set of services may provide a variety of ways to be invoked, with different URLs, ports, and other information distinguishing the various invocations. Each specific collection (single way of invoking a Web service), consisting of a URL, request and response is called a *binding*.

The Web service description is typically generated by the provider and is subsequently parsed by the *requester* in order to configure the requester to invoke the service. In some cases the Web service description may also be used to generate the service.

2.1.2 Request

The request goes through several stages:

1. The caller invokes the service in the requester agent somehow.
2. The requester agent formulates the SOAP message that will invoke the service. It uses the WSDL to create and populate the message body and any headers, attachments, and so forth as set out in the description.
3. The SOAP message is dispatched over the transport (typically HTTP, but not always so) to the URL in the WSDL. This generally represents the provider agent, but may also involve an intermediary server which forwards the request.
4. The provider agent receives and decodes the message. This may result in an error, in which case a SOAP Fault message may be generated as a response.
5. If the SOAP message is decoded successfully, recognized, and valid for a service supported by the provider agent (that is, it was not in error), the provider agent attempts to invoke the service itself.

Note that invoking the service may result in an error (if, for example, the service isn't currently available) and also that the service itself might result in an error.

2.1.3 Response

Not all services generate a response, either when an error is encountered or with a successful result. If the message exchange pattern calls for a response, then the following steps apply:

1. The service returns its result(s) to the provider agent. This may, as noted above, be a successful result or an error.
2. The provider agent formulates the SOAP message that will be the response. The results of the service are used to build any message body and the provider agent fills in any additional header information. If the service generated an error, then the response message will take the form of a SOAP Fault.
3. The SOAP message is then returned over the transport to the requester agent. Note that from the perspective of the provider agent, this may be an intermediary.
4. The requester agent receives the response and decodes it.
5. The results are passed or made available to the client process.

3 Introduction to Internationalization: Definitions for a Discussion of Web Services

3.1 What are Internationalization and Localization?

Users from different countries or cultural backgrounds require software and services that are adapted to correctly process information using their native languages, writing systems, measurement systems, calendars, and other linguistic rules and cultural conventions.

[

Definition: **International Preferences**]The specification of the particular set of cultural conventions that software or Web services must employ to correctly process information exchanged with a user.

[Definition: **Internationalization**]The process of designing, creating, and maintaining software that can serve the needs of users with differing language, cultural, or geographic requirements and expectations.

There are many kinds of international preferences that a Web service may need to offer, to be considered usable and acceptable by users around the world. Some of these preferences might include:

- Natural language for text processing: parsing, spell checking, and grammar checking are examples of this
- User interface language, which may include items like images, colors, sounds, formats, and navigational elements
- Presentation (human-oriented formatting) of dates, times, numbers, lists, and other values
- Collation and sorting

- Alternate calendars, which may include holidays, work rules, weekday/weekend distinctions, the number and organization of months, the numbering of years, and so forth
- Tax or regulatory regime
- Currency
- ... and many more

Because there are a large number of preferences, software systems (operating environments and programming languages) often use an identifier based on language and location as a shorthand indicator for collections of preferences that typify categories of users.

HTML for example uses the `lang` attribute to indicate the language of segments of content. XML uses the `xml:lang` attribute for the same purpose.

Java, POSIX, .NET and other software development technologies use a similar-looking (but not identical) construct known as a *locale* to activate certain internationalized capabilities in software.

[Definition: **Locale**] A collection of international preferences, generally related to a geographic region that a (certain category) of users require. These are usually identified by a shorthand identifier or token that is passed from the environment to various processes to get culturally affected behavior.

Generally, systems that are internationalized can support a wide variety of languages and behaviors to meet the international preferences of many kinds of users. When a particular set of content and preferences is operationally available (often called "enabled"), then the system is referred to as localized.

[Definition: **Localization**] The tailoring of a system to the individual cultural expectations for a specific target market or group of individuals. The target group is often indicated by the [locale](#) identifier.

Localized systems often need to perform matching between end user preferences represented by the [locale](#) and localized resources. This process is called language (or locale) negotiation.

[Definition: **Language Negotiation**] The process of matching a user's preferences to available localized resources. The system searches for matching content or logic "falling-back" from more-specific to more-general following a deterministic pattern.

However, it is important to note that many of the international preferences do not correlate strongly with [locale](#) identifiers based solely on language and location. For example, a system might define a locale of "en-US" (English, United States). This locale encompasses several time zones, so the user's preferred time zone cannot be deduced by the locale identifier alone. Many cultures have more than one way of collating text, and so the appropriate sort ordering cannot always be inferred from the locale. For example, Japanese applications may use different orderings

known as radical-stroke and stroke-radical. Germany and other parts of the world may use different sort orderings known as *dictionary* versus *phonebook*.

Distinguishing these situations requires forethought in the design of the service and the setting of reasonable default values.

Each user or system in a Web services interaction may have its own default [locale](#) settings. The interplay between the requester, provider, service host, intermediaries, and other entities may have complex implications.

3.1.1 Relationship of Locale to Natural Language

There is not yet an Internet standard for

[locale](#) identifiers. However, there is one for natural language identifiers, [\[RFC3066\]](#). Since these language identifiers can imply a locale and in the absence of a standard for locale interchange, language identifiers are often used by software as the source for locale identification. Language and locale are distinct properties and should not be used interchangeably, but there is a relationship between these parameters in the area of resource selection and localization.

The danger of using one for the other lies in the distinction between them. A language preference controls only the language of the textual content, while locale objects are used to control culturally affected (software) behavior within the system. For example, making the assumption that the language parameter *ja* (Japanese) means the data should be presented in the locale-determined format for Japan could be a mistake if the requester actually lives and works in Australia.

The language parameter may be available in several places. In HTTP, there is an Accept-Language header field which can be used (see the HTTP Accept-Language section for more information). MIME has a Content-Language header which contains a language identifier (see the MIME Tags section for more information). In XML, there is an attribute which can be defined for elements called `xml:lang`. `xml:lang` marks all the contents and attribute values of the corresponding element as belonging to the language identified. What that means for processing those contents varies from application to application.

Here are some examples:

```
<p xml:lang="en">The quick brown fox jumps over the lazy dog.</p>
<p xml:lang="en-GB">What colour is it?</p>
<p xml:lang="en-US">What color is it?</p>
<sp who="Faust" desc='leise' xml:lang="de">
  <l>Habe nun, ach! Philosophie,</l>

  <l>Juristerei, und Medizin</l>
  <l>und leider auch Theologie</l>
  <l>durchaus studiert mit heißem Bemüh'n.</l>
</sp>
```

For more detailed information on the behavior of `xml:lang`, see the XML specification.

3.1.2 I-025: Specifying and Exchanging International Preferences in Web Services

Web service and provider implementations, like Web based applications, face the problem of language and

[locale](#) negotiation.

Most Web based application environments have established proprietary standards for performing language and locale negotiation and provide greater or lesser support for managing this form of personalization and content management.

Web services, in contrast, must allow disparate systems to interoperate in a consistent, non-proprietary manner. This design allows systems to invoke each other without regard to the internal architecture of any part of the system. It is helpful to think of a Web service as an remote procedure call ("RPC"), even though many Web services do not use the SOAP-RPC pattern. Unlike Web applications that can store user preferences in a session-like object hidden from the requester, Web service interoperability requires a shared model, if processing is to produce consistency between expectations and result.

Some of the problems inherent in dealing with [locale](#) negotiation and identifiers in Web services include:

Web Service Description Scenario A: A method is implemented in the Java programming language which takes a `java.util.Locale` argument. A Web service description is generated from this method via reflection of the Java class so that the method can be deployed as a Web service. The implementation of the Java `java.util.Locale` class is exposed in the Web service description and requests must be submitted with field values appropriate for Java, which may be difficult or impossible for non-Java clients to provide.

Description Scenario B: The same method is implemented taking a single string argument instead. The programmer creating the method writes logic to translate the string into the appropriate internal locale object. This logic may be substantial and must be repeated or shared for each locale-affected method. There is no way to associate the string argument with locale functionality in the provider, locale or language identifiers available in the transport, or to describe the parameter fully and consistently in directories. A system invoking the service might not be able to create a string in the expected format. The provider may not be able to validate the information appropriately.

Description Scenario C: A existing or "legacy" function or method which obtains its locale information from the runtime environment is deployed as a Web service. Existing locale negotiation mechanisms, such as Accept-Language in many application servers, rely on the container (formerly an Application server, but in this case the service provider) to populate this information. The service provider cannot know that this information is needed. The Web service description doesn't have a

mechanism for describing this environment setting and the results from the service are limited to the runtime default locale of the provider or service host.

Scenario A, Different Locale Identifiers: Sender sends a request to a provider and wants a specific locale and uses its identifier for that. The provider is running on a different platform and doesn't produce the same result as the sender expects.

Scenario A₁, Different Locale Semantics: Sender sends a request to a provider, expecting a result in a specific locale-affected format. The provider has a locale with the same ID, but the specific operation is different from the sender's implementation and the results don't match. These differences are generally subtle, but may vary widely depending on the specifics of the implementation. For example: collation or formatting dates as a string often display subtle variation from one platform to another.

Scenario A₂, Fallback Produces Different Results: Sender requests a specific locale. Provider's fallback produces wildly different results. For example, zh-Hant, the RFC3066 language tag for Chinese written in the Traditional Han script might fall back to zh which represents generic Chinese and, on many systems, implies the use of the Simplified Han script.

The following graphics show some Chinese language tags and the resulting locale object in various systems. Note the differences in interpretation:

Select international context settings:

Language:	zh (Chinese)
Script:	Hant (Han Traditional)
Region:	[not set]
IANA Variant:	[not set]

Show All Fields

Update Display

RFC3066: zh

Java Locale: zh

.NET CultureInfo: zh-CHT

POSIX Locale: zh.UTF8

RFC3066.bis: zh-Hant

webMethods URN: urn:icontext:webMethods:lang=zh:script=Hant

Select international context settings:

Language:	zh (Chinese)
Script:	Hant (Han Traditional)
Region:	TW (Taiwan)
IANA Variant:	[not set]

Show All Fields

Update Display

RFC3066: zh-TW

Java Locale: zh_TW

.NET CultureInfo: zh-TW

POSIX Locale: zh_TW.UTF8

RFC3066.bis: zh-Hant-TW

webMethods URN: urn:icontext:webMethods:lang=zh:script=Hant:region=TW

Here are two additional examples, one for Serbian and another for Azerbaijani:

Select international context settings:

Language:	sr (serbe)
Extended Language:	
Script:	Cyrl (Cyrillic)
Region:	891 (Serbia and Montenegro)
IANA Variant:	[not set]

Show All Fields Update Display

RFC3066: sr-Cyrl

Java Locale: sr_YU

.NET CultureInfo: sr-SP-Cyrl

POSIX Locale: sr.UTF8

RFC3066.bis: sr-Cyrl-891

webMethods URN: urn:icontext:webMethods:lang=sr:script=Cyrl:region=891

Select international context settings:

Language:	az (azéri)
Extended Language:	
Script:	Latn (Latin)
Region:	AZ (Azerbaijan)
IANA Variant:	[not set]

Show All Fields Update Display

RFC3066: az-Latn

Java Locale: ++ Null locale ++

.NET CultureInfo: az-AZ-Latn

POSIX Locale: az_AZ.UTF8

RFC3066.bis: az-Latn-AZ

webMethods URN: urn:icontext:webMethods:lang=az:script=Latn:region=AZ

Scenario B: Sender sends a request to a provider, expecting results in a specific locale-affected format. The sender uses its own locale identifiers. The provider and/or service is on an incompatible platform and cannot interpret the request. For example, converting Microsoft Windows's LCID identifier to Java's `java.util.Locale`.

Scenario C: Sender wants a specific format or set of processing rules for a specific item or set of items. The provider is running on a different platform, so the semantics differ. For example, the sender expects the Java `SHORT` date format, but the provider is written in the C# language.

Scenario D: Sender wants a specific format and sends a picture string or other very specific identifier. The provider and sender must agree on picture string semantics. For example, they must agree on what the picture string symbols stand for. Even in the presence of such an agreement, the underlying data in the different locale models may not match, such as the particular abbreviation for a month name.

Scenario E: Sender wants a specific locale and the provider doesn't support it. This isn't fatal to or detected by the receiving process, which returns data in an unexpected format or with unexpected results. For example, the date May 6, 2004 might be returned in a locale-formatted string as 06/05/2004 and be interpreted by a U.S. English end user as June 5, 2004.

Scenario F: Scenario E, except that it is detected by or fatal to the service. It may be difficult to interpret why the service failed. For example, the date returned in Scenario E might have been 13/05/2004, which is clearly in the wrong format for a U.S. user, but the receiving service may not be able to correct for the problem.

Scenario G: Sender requests results that contain human readable text. The provider returns all languages available.

3.1.3 Locales in Web Service Descriptions

Web service descriptions should consider how to communicate language or

[locale](#) choices in a consistent manner. In the sections that follow, specific patterns are recommended as good canonical references. However experience shows that a specific implementation may require additional contextual information not conveyed with a simple language tag. Generally this type of additional information should be encoded into the message body (that is, as part of the application's design, not as part of the Web services infrastructure). This expresses specific implementation decisions as part of the service's signature: you might require additional or different data in future versions. Some of the examples below show this type of information exchanged in headers and some of the complications that may arise from this.

In the examples below, adoption of a generic method for exchanging "international contextual information" would allow implementations to better model the natural language and locale processing choices offered by the services.

Implementers should consider adding a language tag to any operation fault elements to show what language to expect fault messages to be generated in.

In all cases, descriptive text should be tagged with its actual content language using the `xml:lang` attribute (where permitted). Consideration should be given to providing documentation within services in alternate languages when the service is expected to be utilized by users such as those in other countries or who speak other languages.

3.1.4 Locales in SOAP

Some applications of Web services require a

[locale](#) in order to meet end user expectations. An example of this is any process that returns human readable text messages (many more examples exist and some are given below).

Software developers generally get their messages from language resources using an API provided by the programming environment. This functionality is implemented in many ways, but the pattern for writing the logic is always similar: the language and locale preferences are not included in the parameter list of the service itself because the processing environment (JVM, OS, .NET framework, etc.) maintains this information as metadata about the process or user.

A SOAP Processor implementation might provide accessible natural language or locale preference information, received either in the transport (such as HTTP Accept-Language) or in SOAP headers defined for a particular binding of a service.

For example, a .NET SOAP Processor might set the service's thread default `CultureInfo` using a language tag. A J2EE implementation might populate the `javax.servlet.ServletRequest` class's `Locale` property with a `java.util.Locale` constructed from the ISO639 and ISO3166 fields embedded in a language tag. And so forth.

3.1.5 Faults, Errors, and Human Readable Text

Fault message "text" elements must be labelled with an appropriate language identifier, as defined in XML 1.0. That is, an `xml:lang` tag containing an RFC3066 (or its successor) language identifier. If the transport provides the user's language preference (such as HTTP Accept-Language), then that language or set of languages should be preferred, followed by the SOAP Processor machine's local language preference.

Ideally there should always be a "message of last resort" included in the fault. In many cases this message may be in English, but consideration should be given to the likely users of the system, including the administrators trying to puzzle out the error. Numeric (or ASCII-only alpha-numeric) error codes should be considered for inclusion in all fault messages. This may provide valuable reference when the text of the message itself is in a language not understood by the recipient.

When designing specifications intended for interoperability between vendors or implementations, consideration should be given to enumerating the possible faults in advance so that reference numbers can be universally and consistently referenced by disparate implementations.

3.2 Locale Independent vs. Locale Dependent Data

When designing data structures for applications in general and for Web services in particular, it is important to design data structures in a locale-independent way wherever possible. Keeping the data itself from the representation for the user leads to a clearer application structure, drastically reduces the number of formats for interchange, avoids the need for additional information to distinguish different formats, and allows 'late localization' or 'just in time localization'.

An interesting, informative paper describing late localization is available here:

[\[JITXL\]](#)

The use of XML Schema in Web services helps promote locale-independent data because most of the XML Schema datatypes [\[XMLS-2\]](#) have been designed to be locale-independent.

3.2.1 Textual vs. Binary Representations

In many traditional applications, the distinction between locale-independent and locale-dependent datatypes is also a distinction between binary and textual representations.

As an example, a floating-point number is represented in some binary format internal to an application. It is converted to a textual format when displayed to the user, and appropriate localization is applied to the formatting. For example, it would use a comma rather than a decimal point for many European locales.

Because XML is an inherently textual format, the XML Schema Datatypes also are textual. Nevertheless, most of them were carefully designed to be locale-independent, and are intended to be used in a locale-independent manner.

As an example, the XML Schema Datatype `date` uses the format `YYYY-MM-DD` from

[\[ISO8601\]](#). This format is similar (and in some cases even identical) to some actual formats used in some locales. The format is unambiguous and can be understood by a human reading the XML file. Although it is the appropriate format in some locales and not in others, it can be understood to be a locale-independent format. By contrast, if XML Schema had chosen a format that is not used in any locale, such as just numbering days since a well-defined day, it would have made the format much more difficult for humans to work with, without any benefits.

3.2.2 Locale-Dependent XML Schema Datatypes

While most datatypes in XML Schema are locale-independent, there are a few that are locale-dependent, and therefore should be avoided. These are all the datatypes that start with 'g', namely `gYearMonth`, `gYear`, `gMonthDay`, `gDay`, and `gMonth`, and the `duration` datatype.

The semantics of these datatypes are bound to the Gregorian calendar. As an example, a field of type `gMonth` with a value set to 5 refers to the month of May in the Gregorian calendar. This concept cannot be converted to calendars that do not

have their months aligned with the months of the Gregorian calendar, such as the Islamic, Hebrew, or Ethiopic calendars.

As another example, a `gMonthDay` field with value of `09-12` refers to September 12th in the Gregorian calendar. This date may coincide with the first *or* the second day of the month *Meskerem* in the Ethiopic calendar depending on the year.

On the other hand, the semantics of the other date- and time-related XML Schema datatypes are not bound to the Gregorian calendar, although they rely on the Gregorian calendar for their lexical form. For example, the date `2004-09-12` can not only be converted to September 12, 2004 (using the Gregorian calendar), but also to the first day in the month *Meskerem* in the year 1997 in the Ethiopic calendar (the Ethiopic New Year).

3.2.3 Examples

Example: Some locale-independent XML Schema datatypes	
TYPE	: EXAMPLE

<code>date</code>	: 2003-05-31
<code>time</code>	: 13:20:00
<code>dateTime</code>	: 2003-05-31T13:20:00+09:00
<code>double</code>	: 1267.43233E12
<code>integer</code>	: 2678967543233

Example: Locale-dependent XML Schema datatypes	
TYPE	: EXAMPLE

<code>duration</code>	: P1Y2M3D
<code>gYearMonth</code>	: 2003-05
<code>gYear</code>	: 2003
<code>gMonthDay</code>	: 05-25
<code>gDay</code>	: 25
<code>gMonth</code>	: 05

4 Basic Web Service Internationalization Scenarios

4.1 Locale Patterns in Web Services

Distributed processing, as with Web services, must allow for several patterns of behavior in the service.

There are four general patterns or policies that may be applied to the various aspects of a Web service. These four are:

- Locale Neutral
- Client Influenced

- Service Determined
- Data Driven

4.1.1 The Travel Application

This section uses examples from the travel industry to illustrate all four patterns.

4.1.2 Locale Neutral

In the *Locale Neutral* pattern, the service executes in the same locale-independent way regardless of how it is deployed, invoked or configured. Locale neutral services generally do not provide culturally sensitive processing or deal with human-readable natural language text.

An example of this would be a service that returns flight arrival time using the XML Schema datatype `xsi:dateTime`. This data type is expressed in a universal format. It is then up to the requester to decide if and how to process or format the results.

This means that the strings embedded in the SOAP message produced from the service's results are intended to be machine readable and that data processed by the service does not need to be handled in a culturally sensitive way. The requester might format the value for display to human end user, but this is not part of the service's lifecycle.

Locale neutral services, being the default, do not generally need to announce their locale in their service contracts, although information about the locale or language preference of the requester may still be useful to the service or provider (for example in generating fault messages for diagnostic purposes).

4.1.2.1 Example: 'GetArrivalTime' Returns Flight Arrival Time

A service to get flight arrival time 'GetArrivalTime', can be written in a locale-neutral way.

For example, the service response would contain a single value `arrivalTime` using the current UTC (Coordinated Universal Time) time in the ISO 8601

[\[ISO8601\]](#) format: `YYYY-MM-DDThh:mm:ss.sss`, mandated by the `time` datatype in XML Schema Part 2: Datatypes[\[XMLS-2\]](#).

Any requester can transform the result into a local time format, including shifting the time into the local time zone. This way the requester agent, service provider, the service, and the result are entirely independent of the locale of the client, the host, and the implementer. Hence the service is locale-neutral.

By contrast, a service that returns a locally formatted string containing the arrival date and time should be dependent on the locale and language preferences of the requester.

4.1.3 Client Influenced

In the *Client Influenced* pattern, the service provides a specific set of localized behaviors which are tailored according to the

[locale](#) preferences of the requester.

The service must provide a way for the requester to communicate the preferences, and, if there is a response, it should communicate the actual value used to perform the processing.

This pattern's name uses the term 'influenced' because the provider and service may not have all possible languages, locales, or sets of preferences available as resources. The service might perform language negotiation and 'fall-back' to a more general set of preferences or use its own preferences if the preferences requested cannot be satisfied.

4.1.3.1 Example: 'getItinerary' Get Flight Information in the Requester's Language

Textual information is returned to a requester by a service for display to a human end user. This might include information such as fare rules, luggage limitations, etc., that the end user would like in their own language.

Therefore, the requester sends language preferences information to the provider, which invokes the service. The service looks up and returns the requested textual information in the requested language.

```
<flight>
  <airline>XA</airline>
  <flight>116</flight>
  <restrictions xml:lang="en">Change requires
    USD100.00 fee. Bags limited to two plus one
    carry on.
  </restrictions>
  <departs>
    <code>SF0</code>
    <name xml:lang="en">San Francisco</name>
    <date>2004-04-15</date>
    <time>16:45-0800</time>
  </departs>
  <arrives>
    <code>AER</code>
    <name xml:lang="ru">Сочи</name>
    <date>2004-04-17</date>
    <time>23:45+0300</time>
  </arrives>
</flight>
```

4.1.3.2 Service Description

Locale negotiation is required to enable this pattern. The request must allow at least one preference and should allow multiple preferences to be passed, in case the first preference is not available for this service. The provider's response should return the value(s) actually used by the service.

There are several ways to accomplish this. For example, the service can be written to include this negotiation in its implementation. This requires the service to include the locale preference as an argument in the service's parameter list.

4.1.4 Service Determined

In the *Service Determined* pattern, the service provides a specific localized set of behaviors determined prior to the request.

That is, the service does something in a culturally specific or locale affected way and the locale used in this processing is inherent in how the service is implemented, installed, deployed, or configured. The major difference between this pattern and the client influenced pattern is that the service and provider do not perform language negotiation for whatever reason.

For example, the service might use the default locale of the system where the provider is running.

It may also be a configuration option of the service, provider, or provider agent. One example of this would be a service that provides several bindings, each with a separate locale.

The preferences of the requester therefore do not influence how the service executes and may not influence aspects of the service such as the language of messages returned and so forth.

4.1.4.1 Example: 'flightCheck' Service

A simple example of the service determined pattern could be a service that returns the current flight status for particular airport.

The service takes arguments of `xsi:date` for the flight date, `xsi:string` for flight number as input and returns an `xsi:string` with the current flight status. The message returned is in the language where the airport is located.

A German system, for a flight from Hamburg to Tokyo might include a text note, "ein Tag später" to indicate that the flight arrives one day later.

The service determined pattern may apply to a service for a number of reasons. Some of these include:

- *Implementation Decision.* The service's code may be written with a hard-coded locale.

```

public static void myService(String input) {
    Locale serviceLocale = Locale.GERMANY;
    DateFormat df = DateFormat.getDateInstance(serviceLocale);
    // ...
}

```

- *Service Host Configuration.* The service's code may use the system default locale or may not use international APIs that allow the programmer to control the locale.

```

public static void myService(String input) {
    Locale serviceLocale = Locale.getDefault();
    DateFormat df = DateFormat.getDateInstance(serviceLocale);
    // ...
}

```

```

public static void myService(String input) {
    DateFormat df = DateFormat.getDateInstance();
}

```

- *Provider Configuration.* The provider or provider host may be configured so that the service's execution environment has a particular locale. In many cases, the provider and the Service Host may be the same thing.

```

public static void myService(String input) {
    Locale serviceLocale = // call provider specific API to return a
    DateFormat df = DateFormat.getDateInstance(serviceLocale);
    // ...
}

```

- *Deployment Decision.* The same service might be deployed with different endpoints (URIs) that represent different locales.

The container server could also influence how the service executes (if the service's implementation merely reads the default locale from the runtime or obtains it from an API).

Example: Message portions of WSDL for 'flightPrice'

```

<wsdl:message name="flightCheck0In">
  <wsdl:part name="arg0" type="xsd:date"/>
  <wsdl:part name="arg1" type="xsd:string"/>
  <wsdl:part name="arg2" type="xsd:int"/>
</wsdl:message>

      <wsdl:message name="flightCheck00Out">
<wsdl:part name="Result0" type="xsd:decimal"/>
<wsdl:part name="Result1" type="xsd:string"/>
</wsdl:message>

```


4.1.5 Data Driven

In the *Data Driven* pattern, the service's performance depends on the locale of the data (or, more usually, the data source or back-end system) itself, rather than the requester, provider or the service itself.

As an example, a service dealing with flight schedules will use the time zones of the respective departure and arrival locations for departure and arrival times, rather than some server-related time zone or the time zone preference of the client. The ISO 8601 format used by XML Schema might express times using the

[\[RFC822\]](#) UTC offset for each time zone, rather than attempting to use a single time zone in the messages.

As another example, a service that queries a database might return data sorted or selected based on the database's configuration, rather than an external setting (such as the requester or service locale).

4.1.5.1 Example: 'getWeightRestrictions' Gets Flight Luggage Restrictions

In this case, the application retrieves information that is dependent on the stored flight information. In the U.S.A. it would be in pounds. In most other parts of the world it would be in kilograms.

4.1.5.2 Example: Stored User Preferences

If a user travels to a remote location or uses a different system to access an application than they normally would (by visiting an Internet cafe while on vacation, for example), the local system won't have the user's preferences stored locally. The user could supply credentials, such as a login, to the application, which uses it to obtain user metadata (such as a language preference stored in an LDAP directory) to control the responses sent to the user. Thereafter, text and locale-specific information can be formatted on the provider side for inclusion in SOAP messages sent to the requester. The requester can be notified by the provider what the language preferences of the user (or process) are, and the requester could then adapt how data is displayed accordingly.

4.1.5.3 Example: Data from Service to Service

This final example is really a permutation of the Service Influenced model, but in this case, a Requester "A" requests services from a Provider A, which then requests services from Provider "B". Provider "B" either has a fixed locale, or dynamically sets the locale based on information given it by a service requester. That information is then propagated back to the original Requester "A".

4.2 Locale and Language Dependency in Message Exchange Patterns

When exchanging messages, the requester and the service that the requester accesses may have different default locales and language preferences. In addition, there may be more than one service involved in the message exchange or there may be different clients who consume the message. Each of these may expect a different locale and language.

Message exchange between components with different language and/or locales may result in a failure or unexpected result. This section describes various message exchange patterns that need to consider language preferences or that have potential failure scenarios.

4.2.1 I-009: One Way Messages

One-way messages that do not have a response may still have language-related issues.

1. Service A is defined to receive a message from Requester A and deliver to Requester B via Service B. An example of this would be similar to a mail server, message queue, or messaging broker triggered by a Web service.
2. Requester A calls Service A.
3. Service B is unable to complete its message transaction and generates a fault.

Service B should return a message in a language that matches Requester B's language preference (so that the administrator of that system can use it). In addition, if Requester A's preferences are available to Requester B (that is, Service A got the preferences as part of its input or via an external mechanism such as the transport), then Requester A's language preference should be included in the SOAP fault reason text.

Example: Message Relaying Fault

- Requester A has a language preference of fr-FR
- Provider B is running in an environment with a language preference of de-DE
- Service B is running in an environment with a language preference of en-US

```
<env:Reason>
  <env:Text xml:lang="fr-FR">erreur en français</env:Text>
  <env:Text xml:lang="de-DE">Verarbeitungsfehler</env:Text>
  <env:Text xml:lang="en-US">Processing error</env:Text>
</env:Reason>>
```

4.2.2 I-018: Data Associated with a Default Attribute

Default attributes can cause difficulties in Web services, since they often do not travel with their corresponding values. For example, a provider deploys a Web service that returns currency values as a single numeric field, without identifying the currency. For example, the underlying function could assume that all currency amounts are returned using the euro currency, whose ISO 4217 currency code is EUR. Responses from this service are used by multicurrency services; since the default currency type of 'euro' doesn't accompany the numeric values, these other services may make assumptions about which currency applies to those values. This could cause problems with incorrect monetary values going out to requesters.

A solution to the above problem is to include the currency value in the message body which corresponds to the numeric value. This solution, however, may not be viable for service providers that are wrapping existing functions or which generate the Web service description via introspection. In these cases, the service provider may need to associate a 'default attribute' with the messages generated by the underlying service. This would allow requesters to recover the currency information.

The following example demonstrates multiple currency data transmission in a SOAP message and the currency code being provided in a separate element along with the value. A currency data structure such as the one shown below is one suitable solution for internationalized Web services.

Example: Currency Object: Sample Web Service Description

```
<wsdl:definitions name="currencyExample" targetNamespace="http://www.example.com"
  <wsdl:types>
    <xsd:schema targetNamespace="http://www.example.com/package/">
      <xsd:import namespace="http://schemas.xmlsoap.org/soap/encoding/" />
      <xsd:import namespace="http://www.example.com/something/" />
      <xsd:complexType name="currencyExample$Price">
        <xsd:all>
          <xsd:element name="c" minOccurs="0" nillable="true" type="xsd:float" />
          <xsd:element name="value" minOccurs="0" nillable="true" type="xsd:float" />
        </xsd:all>
      </xsd:complexType>
    </xsd:schema>
    <xsd:schema targetNamespace="http://www.example.com/something/">
      <xsd:complexType name="Currency">
        <xsd:all>
          <xsd:element name="currencyCode" minOccurs="0" nillable="true" type="xsd:string" />
          <xsd:element name="value" minOccurs="0" nillable="true" type="xsd:float" />
        </xsd:all>
      </xsd:complexType>
    </xsd:schema>
  </wsdl:types>
```

Note:

Currency codes are widely standardized using ISO 4217, which provide alpha-3 codes for each currency. The service must, of course, provide validation logic for the `currencyCode`.

Example: Multiple currencies in a SOAP message

```
<?xml version='1.0' encoding="UTF-8" ?>
<env:Envelope xmlns:env="http://www.w3.org/2002/06/soap-envelope" >
  <env:Header>
  </env:Header>
  <env:Body>
    <c:purchase>
      <c:apple>
        <price>
          <currency>
            <currencyCode>USD</currencyCode>
          </currency>
          <value>4.56</value>
        </price>
      </c:apple>
      <c:orange>
        <price>
          <currency>
            <currencyCode>JPY</currencyCode>
          </currency>
          <value>442</value>
        </price>
      </c:orange>
      <c:peach>
        <price>
          <currency>
            <currencyCode>EUR</currencyCode>
          </currency>
          <value>3.97</value>
        </price>
      </c:peach>
    </c:purchase>
  </env:Body>
</env:Envelope>
```

Adding parameters to the SOAP body requires design changes to the service interface and possibly to the implementation. Adding default values into SOAP headers does not affect the service interface and often can be done statically for a particular resource. This may be an acceptable solution when presenting data from legacy systems through Web services. For example, this could be used for adapting a legacy retail or banking system which conducts all transactions in a single currency to provide data to an international system, however there are many potential issues with this design (see section

[4.4.1 Pandora's box: Using Non-internationalized Data Structures](#)).

Example: Wrapping a non-internationalized service using a default header value:

```

<?xml version='1.0' ?>
<env:Envelope xmlns:env="http://www.w3.org/2002/06/soap-envelope" >
  <env:Header>
    <example:SomeInternationalization xmlns:example="http://example.org/"
      <example:Currency>JPY</example:Currency>
    </example:SomeInternationalization>
  </env:Header>
  <env:Body>
    <c:purchase>
      <c:apple>
        <c:price>123</c:price>
      </c:apple>
      <c:orange>
        <c:price>3250</c:price>
      </c:orange>
      <c:peach>
        <c:price>3612</c:price>
      </c:peach>
    </c:purchase>
  </env:Body>
</env:Envelope>

```

4.2.3 I-013: Conflicts Between Requester's Expectations and Service's Locale

In some cases, the locale in which the service provider is executing could affect how parameters sent are evaluated and lead to faults that the requester does not expect or does not understand.

1. Service A is defined on Provider A, running in a Finnish locale, which queries a company's credit records.
2. Requester A is running in a French locale. The Requester is doing a monthly audit to verify local customer credit. To request the credit information from the Finnish provider in manageable chunks, the requester makes multiple requests in which it requests that the customer records fall into a range.

If one were to write the query in SQL, it might take the form: `SELECT * WHERE (LASTNAME >= low) AND (LASTNAME < high)` iterating over the variables `low` and `high`.

3. The first iteration sends a low value of "A" and a high value of "B".
4. Requester A compares the results with its list of customers in that range. The audit is missing many customer records, since the Finnish provider doesn't consider names beginning with either "Å" or "Ä" to be less than "B" but the French requester expects this.

4.3 Fault Handling

SOAP Version 1.2 allows the provider to send fault messages that provide a description of the *reason* the service failed in multiple languages. SOAP Version 1.2 Part 0: Primer

[\[SOAP-0\]](#) explains the <Reason> element as follows: "It must have one or more env:Text sub-elements, each with a unique `xml:lang` attribute, which allows applications to make the fault reason available in multiple languages. (Applications could negotiate the language of the fault text using a mechanism built using SOAP headers; however this is outside the scope of the SOAP specifications.)"

This mechanism is suitable for returning faults in an environment in which the number of languages is relatively small and the range of languages to be returned is known in advance.

SOAP implementations are often localized into many languages simultaneously. To prevent faults from becoming overly large and difficult to manage, implementations should include some strategy that reduces the set of languages returned to those of interest to client(s). This requires a mechanism to match the language of the fault as closely as possible to the client's preferences.

Internationalization best practice is to perform late localization, in which messages are formatted or resolved to strings as late as is reasonable in a process. This preserves language independence and flexibility in responding to multiple users with different language or cultural needs.

Future versions of SOAP should probably consider allowing additional structured information in a Fault so that suitably internationalized clients can perform the localization and formatting themselves.

4.3.1 I-004: Producing Fault Reasons in All Available Languages

In the absence of language negotiation, the service provider may need to produce fault messages in all available languages.

A service provider detects an error in a request. The provider generates a response containing a SOAP Fault. In the absence of language negotiation, the provider must return several, if not all, available languages. The provider cannot know what languages (resources) are installed, since most programming environments do not provide functionality that enumerates which languages are actually installed.

If the service provider wishes to return all languages available, it must request every possible installed locale in turn in order to poll whether the string is available in that language. Since resources are often sparsely populated, this suggests significant a processing overhead to loop over all possible locales, loading resources in turn.

If the service provider returns a significant portion of the languages available, the size of the Fault message may also be adversely affected.



Example: Number of Locales in Selected Operating Environments Default Installation

```
Solaris 8: 42
Redhat Linux 497
SuSE Linux 9.0: 227
Java JDK 1.4.2: 150
Microsoft .NET: 120
```

4.3.2 I-005: Language Matching for Fault Reason Messages

The service requester needs to select a matching language from the list of fault reasons returned by the service provider. Language tag matching and language ranges are described by RFC3066

[\[RFC3066\]](#). Since the `xml:lang` value associated with the Reason Text element may not be empty, the requester may be unable to match any of the returned text elements to its current end user language.

RFC3066 language tag matching and SOAP Reason Text elements do not provide for a default message: there is only a list of different language messages. So the requester must choose some reasonable default from the list of messages provided.

Example: SOAP fault reason messages in multiple languages

```
<env:faultReason>
  <env:Text xml:lang="en-US">Processing error</env:Text>
  <env:text xml:lang="cs">Chyba zpracování</env:Text>
</env:faultReason>
```

If the requester prefers `en-GB`, then neither string will match directly for the current requester language preference. Although it is apparent to a human that `en-US` is a reasonable match for `en-GB`, automated processes are not permitted to make the assumption that languages with common prefixes are mutually understandable.

If the requester prefers `ja`, then selecting the best fallback is even more difficult, since there is no matching language. How can an implementation choose which message to display?

4.3.3 I-008: Locale Sensitive Formatted Data in SOAP Fault Messages

A service or service provider substitutes locale-sensitive data into text messages when generating fault reasons.

A service generates a fault during invocation, resulting in a SOAP Fault being returned to the requester. In order to properly present values in the Reason element of the Fault, the locale must be known and locale information must be available. For example:

- "The date provided, 12 November 2201, was too late."
- "The argument 12345.678 was too large."
- "The argument 12345,678- was too small."

The service or provider should format the substitutions in each message according to the language and locale of the target audience (typically the requester), which is not necessarily the same as the locale of the provider or service.

If locale or language preference information is not available from the requester, it may not be possible to generate a message in the user's preferred language. In these cases, the message should follow the language preference of the provider or service host.

Note:

For additional examples of locale-affected formatting in Web services, see

[I-022](#)

4.4 Legacy Issues

These scenarios cover internationalization issues that arise when moving existing applications to use Web services.

4.4.1 Pandora's box: Using Non-internationalized Data Structures

A data structure may be provided without international considerations. This may happen, for instance, when a service was originally designed and targeted for a specific local market and later converted to a Web service.

Use and adoption of locale-affected and non-internationalized data structures generally requires a firm understanding of the conventions being adopted and careful implementation. It is generally poor practice in Web services, since the goal of having a Web services infrastructure is usually interoperability without deep knowledge of the underlying implementation decisions.

This is an example of a user's daily activity provided in Japanese 12 hour time scheme.

Example: My schedule

Time	:	To do
GOZEN 8:00	:	Breakfast
GOGO 0:00	:	Lunch
GOGO 7:00	:	Dinner
GOZEN 0:00	:	Go to bed

GOZEN means "before noon", and generally corresponds to AM. GOGO means "after noon", and generally corresponds to PM. The problem is GOGO 0:00 is noon rather than 0:00 AM, and GOZEN 0:00 is midnight rather than 0:00 PM. This is confusing and conversion to an internationally known time format may fail.

4.4.2 I-019: Locale Dependent Datatypes

A sender wishes to send locale dependent data to a receiver. The receiver needs to process the locale dependent data correctly.

As an example, if a Japanese sender sends dates to a Japanese receiver, the Japanese sender may wish to send the data in a Japanese date format as required for government records, such as H13-5-31(H stands for the **Heisei** era; see Appendix

[C Heisei](#)).

Example: Locale sensitive data in regional datatype

```
<?xml version='1.0' ?>
  <env:Header>
    <example:WSinternationalization
      xmlns:example="http://example.org/2002/11/21/WS-I18N">
      <example:datatypePreference>
        <ja:JDate xmlns:ja="http://example.org/2003/12/3/ja">EYY-MM-DD</WS-I
      </example:datatypePreference>
    </example:WSinternationalization>
  </env:Header>
  <env:Body>
    ...
    <JDate:birthDate>H13-05-31</JDate:birthDate>
    @@Note: We need to figure out how to properly present this data type
    ...
  </env:Body>
```

Wherever possible it is best to avoid locale dependent data types. Otherwise, the sender and receiver must agree in advance on the data formatting and semantics.

Many locale- or region-specific data types such as telephone number, address elements, post code, forms of address, etc. can be modeled as strings with patterns. It is the responsibility of the application on either end to validate, interpret, and format the data properly.

WSDL and SOAP can be used to constrain locale- or region-specific data fields.

Applies to: WSDL, SOAP, or Localizable datatype

4.4.3 Existing Web Services

If a Web service is created from a specific agent which is not internationalized and later the agent is internationalized, then the service itself may have to be redeployed because the parameter list, headers, or other information have

changed. The developer of the agent is responsible for supplying the fields, logic, and semantics that will be used to achieve international capabilities.

The international considerations in doing this include:

- What information is needed to enable international operation.
- What fields and format are needed to collect that data.
- Fallbacks, validation and whether the items are optional or not.

If the two versions of the service vary in their approach or don't supply a suitable mechanism then the service itself may have changed enough....

Scenario A: There is a Service A which is not internationalized. Some senders start using this service. Later a new version is created that is internationalized, B. Service B cannot simply replace Service A because the service signature has changed. Service A must remain deployed until all users have switched.

Scenario B: There is a Service A which is not internationalized. Later a new internationalized version, Service B, is created that is internationalized but which uses the same fields and required values as Service A. Senders that used Service A may be switched to Service B without warning. Now the behavior may differ... (users expecting Service A may get different results from what they expect).

Scenario C: There is a Service A which is not internationalized. Later a new version is created that adds optional fields to enable internationalized behavior and this new service is deployed as Service B, replacing Service A. Existing senders to Service A do not notice the change, but new senders can use the optional fields to get internationalized results. This may be a way to avoid the problems with Service B, but may be difficult to maintain over time.

Scenario D: Service B is deployed in place of Service A. The new service either generates faults (to deprecate the original invocation) or acts as an intermediary that invokes the original Service A at some new location. Service B can then record senders that are using the original service as a way of assisting in migration.

4.5 Character Encodings and Web Services

SOAP interactions rely on being able to exchange data in a consistent, mutually understandable way. The character encoding of the SOAP message and the communication of the encoding between senders and receivers enable this to occur reliably. Because all XML

[\[XML\]](#) processors must be able to read entities in both the UTF-8 [\[RFC3629\]](#) and UTF-16 [\[RFC2781\]](#) encodings, using UTF-8 or UTF-16 guarantees character encoding interoperability on the SOAP layer. The Character Model for the World Wide Web [\[CHARMOD\]](#) document describes these considerations and guidelines.

4.5.1 SOAP Documents and the MIME Charset Parameter

Scenario Definition: A SOAP interaction is based on SOAP 1.1. The documents exchanged use the Content-Type 'text/xml' without a charset parameter. The default character encoding for the 'text/xml' media type is US-ASCII, so omitting the charset parameter may result in failure or data loss.

The charset parameter must be supplied in order to ensure correct interoperability.

A provider or requester is using SOAP 1.2, which uses the Content-Type 'application/soap+xml'. If the charset parameter for that media type is omitted using 'application/soap+xml' then the SOAP document will be examined for its character encoding using the rules provided in XML.

Note:

In all cases the charset parameter in the media type takes precedence over that of the XML that forms the SOAP document. Please refer to RFC3023, XML 1.0, and RFC2045/2046 for more information.

4.5.2 Character Encoding of Attachments

Frequently Web services have attachments. Since attachments are not part of the SOAP document, they can be in a different encoding. In order for an attachment to be processed correctly, the encoding must be identified for the processing application.

The SOAP processor (either in the provider, in the case of receiving a SOAP request, or in the requester's client when receiving a response) must handle the attachment correctly. For textual data types, this means providing a correct charset parameter in the MIME type when generating the SOAP message. It also means that the recipient of an attachment must provide this information to the ultimate processor (such as the service or the client). Loss of the encoding may result in corruption of the data or having to guess.

If the receipt of the attachment and its processing are separated in some manner, it may be important to provide the encoding of the attachment as part of the actual SOAP message type. This way the text can be processed without access to the MIME header information.

For example, a Web service takes the title of an article and retrieves the full text of the article as an attachment. The database contains articles in many languages and character encodings from sources around the world. The articles are in plain text, with no internal encoding identifier, but when stored in the database, the encoding is inserted as a related parameter. The return SOAP message contains an attribute with the encoding.

```
MIME-Version: 1.0
Content-Type: Multipart/Related; boundary=MIME_boundary; type=text/xml;
              start="<article123456@samplerepository.co.uk>"
Content-Description: Sample SOAP envelope with attachment in ISO-8859-15.
```

```

--MIME_boundary
Content-Type: text/xml; charset=UTF-8
Content-Transfer-Encoding: 8bit
Content-ID: <article123456@samlerepository.co.uk>

<?xml version="1.0" encoding="UTF-8" ?>
  <env:Envelope xmlns:env="http://www.w3.org/2002/06/soap-envelope" >
    <env:Header>
    </env:Header>
    <env:Body>
      <encExample:articleRetriever>
        <articleTitle xml:lang="fr">Voici le titre</articleTitle>
        <articleEncoding>ISO-8859-15</articleEncoding>
        <articleSize unit="byte">14355</articleSize>
      </encExample:articleRetriever>
    <!-- ... -->
    <att:SecondaryPartBag>
      http://samlerepository.co.uk/retrieval/database/article123456
    </att:SecondaryPartBag>
  </env:Body>
</env:Envelope>

--MIME_boundary
Content-Type: text/plain; charset=ISO-8859-15
Content-Transfer-Encoding: 8bit
Content-ID: <article123456@samlerepository.co.uk>

...plain text article in ISO-8859-15...

--MIME_boundary--

```

Note that the SOAP message itself is in the UTF-8 encoding, which is different from the attachment's encoding of ISO-8859-15.

4.5.3 Unsupported Charset in Request Scenario

A requester sends a SOAP document in an encoding that the provider does not support. The provider should fail and return a fault.

4.5.4 Unsupported Charset in Response Scenario

A requester receives a response. The response is encoded using a character encoding not supported by the requester. This is an unrecoverable error. Web services providers and requesters should agree in advance on the collection of encodings that will be used in the transactions. Ideally all transactions will use a Unicode encoding, such as UTF-8, since all XML processors are required to handle this encoding.

4.5.5 Unsupported Characters

Some encodings have more characters than are included in Unicode or use Private Use characters. SOAP messages sent using these problematic characters may result in transient failure or odd results. These characters should be avoided

wherever possible or some mutually agreed upon private solution should be adopted. Note that this is a rare problem.

4.5.6 Legacy Application Use of Non-Unicode Character Encodings

One use of Web services is to wrap existing or legacy systems. These systems may use a legacy (non-Unicode) encoding internally or in their API. Although the service provider supports Unicode for sending and receiving SOAP messages, the underlying system does not. This may lead to:

4.5.6.1 *Calling the Service Requires Transcoding*

The service provider receives and processes a SOAP message. The processor invokes the service, which uses a legacy encoding. Data may be lost or corrupted by the transcoding process between the receiving SOAP processor and the agent. The transaction may seem to succeed, even though the data is corrupted.

4.5.6.2 *Service's Internal Implementation Performs Transcoding*

A service may perform transcoding internally, either as a function of its work or due to the use of backend data storage or repositories configured to non-Unicode encodings. The transaction may seem to succeed, even though the data is corrupted, or it may experience transient errors.

A Web service "insertCustomerRecord" is created for a relational database. The database uses ISO-8859-1 as its internal encoding. The new record sent by the requester contains all Japanese characters (which are not supported by the ISO-8859-1 encoding). The invocation of the service succeeds, even though all of the Japanese characters are converted by the transcoding process to the substitution character (generally a ?). The failure may not be detectable except by inspecting the resulting data.

Additional problems may result. For example, it may be hard to find or remove the resulting data records created by internal transcoding of this nature. In addition, the database may report unexpected key collisions when inserting other records (because portions of both records have been converted to question marks).

4.5.7 Variability in Transcoding Scenario

Note that the XML Japanese Profile

[\[XML-JP\]](#) states that using legacy encodings such as Shift_JIS cannot provide complete interoperability in information interchange; there are differences among platforms in the mapping tables they use for this and similar encodings.

4.6 Passing or Matching International Preferences

International preferences, which may include language, locale, collation, time zone and other preferences, may be passed between systems in a variety of ways. Since most of these preferences are not standardized, implementers must define messages or complex types using XML Schema for their specific needs.

The values available to requesters for the preferences and the matching algorithm in the service (that tries to select the functionality or content based on the values passed) must be set up to deal with fallbacks and suitable defaults.

Implementers may also have to define internationalized behavior beyond that described by a mere language or locale choice. It is common for these design decisions to be specific to the particular application or particular market being serviced.

An airline flight reservation system might concern itself with details of personalization such as smoking or meal preference, window or aisle seating preference and the like. While these personalization elements are clearly not the realm of a locale or international preference, other items may not be as clearly delineated.

The same airline reservation system might need to be concerned with local regulations for different airports or with varying time zones.

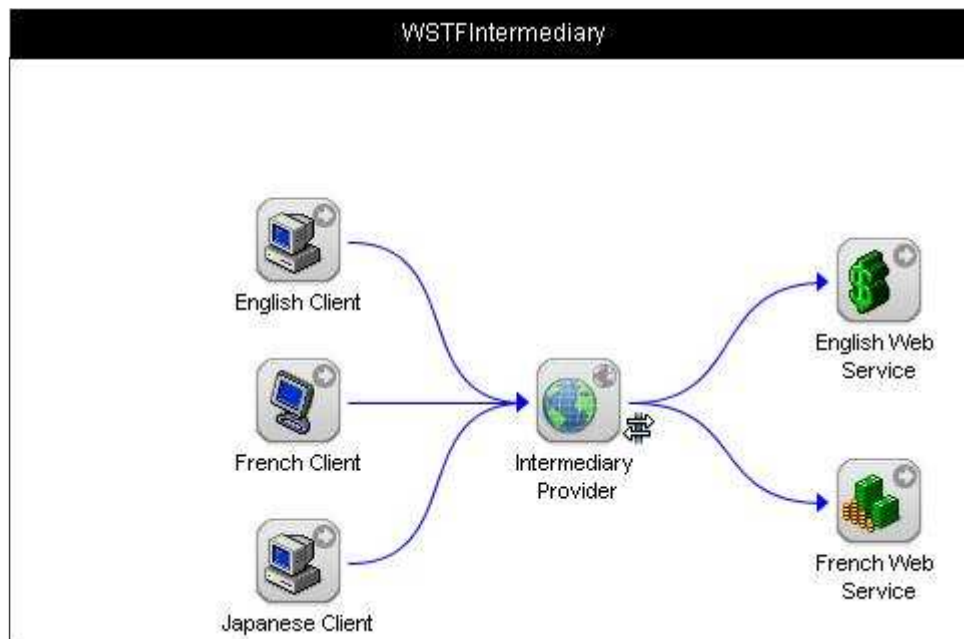
The use of a locale or language preference as a short hand for these more complex requirements should be carefully considered, and possibly discouraged, in favor of making the specific information required for proper operation explicit in the service contract.

Nonetheless, in some cases the service implementer may wish to use the language or locale preference of the end user to determine how the service's processing should proceed.

4.7 Intermediaries and Internationalization

Scenarios in this section deal with issues that arise when services employ intermediaries, such as those discussed in "Service Oriented Architecture Derivative Patterns Intermediary" (in Web Services Architecture document

[\[WSA\]](#)).



As the diagram indicates, one or more providers offer services. An intermediary provider can deploy a service that makes requests from these providers and uses the results to satisfy the requests coming from its clients. The intermediary service may process and/or integrate the results from different providers to create a new kind of service or it may simply pass results along. The intermediary service may also cache either the contents it sends to clients, or the results returned to it by its providers, for reuse with subsequent requests. In these scenarios it is important to consider that the providers may return results formulated for certain international preferences. Clients may also be expecting results formulated according to their specific requirements. The intermediaries may be expected to apply appropriate matching between client and provider, or to bridge gaps.

4.7.1 I-020: Correlation of Data Between Services in Different Languages

Clients requesting from the intermediary service can have different international preferences. Therefore the intermediary service must be careful with its algorithms for determining when to reuse responses that it receives from its providers. Proper tracking of source data locale and requester locale is required. Also, correlation and/or aggregation of data may prove difficult if sound internationalization principles are not used.

4.7.2 I-007: Interaction of Optional Locale and Language Negotiation and Chained Services

If caching does not take international preferences into account, it is possible that cached responses in the wrong language, format, or locale could be returned.

Alternatively, in scenario I-020, the intermediary service caches fault reasons and other data returned from its providers in each of the languages and cultural conventions that are requested of it, tracking the locales of each result.

Requesters of the intermediary service identify the desired locale of expected results. With locale negotiation, the intermediary service can provide results and/or fault reasons that match the requester's international preferences.

4.7.3 I-012: Caching

Chained services are a form of intermediary services. A (source) provider defines a service that has a requirement for a language or locale preference. Another (intermediary) service provider, defines the same service and invokes the first service to utilize its capabilities.

The source provider defines an optional header containing a language request field. If the intermediary service does not also define the optional header, then when it receives a request it cannot communicate the requester's preferences to the source provider. The intermediary service might indicate its own international preference(s) to the source provider or none, accepting default values. Unless, the description of the intermediary's service declares its policy on addressing international preferences, its users may have incorrect expectations of the results.

4.7.4 Caching with Locale Information in SOAP Headers

In a variation of the above scenarios, a SOAP header can be used for locale negotiation between each layer of requester, intermediary and service. An example is a Web service wrapper to a legacy client/server application.

If the intermediary strips off optional headers (either from the requester of its service or the response from its providers), some of the contextual information necessary to interpret the response may be lost.

4.8 SOAP Header Structures

The SOAP header is an optional element which can be used to extend SOAP processing in an application-specific manner. The header specification is intentionally minimal so that headers may be tailored to meet the needs of various applications.

SOAP headers may be used to initiate or control processing of the message data, either by the ultimate receiver of the message or by intermediary nodes which handle the message before it is routed to the ultimate receiver. In this role, SOAP headers may contain information specifying the routing of SOAP messages and the processing which may (or must) occur at intermediary nodes.

Headers may be used to convey additional contextual information about the data in the body of the SOAP message. In the context of internationalization, although applications are encouraged to use locale-neutral data formats, processes, and methodologies, in locale-sensitive scenarios, the SOAP header could be used to declare the locale to be associated with the SOAP message. Of course, this technique can be extended to other culture-dependent information that is not prescribed by the locale. (For example, a SOAP message with shoe size data

might require a SOAP header to declare the shoe measurement system that is used.)

Of course, generally, it is preferable to include this kind of contextual information directly with the data. Currency is an example of this: it is preferable to name the monetary unit with the amount. There is a risk when contextual information is maintained separately from the data (for example by placing contextual information in a header) that modifications will be made to one without appropriate changes to the other.

Another risk is that the message content references data from more than one locale. This creates a complex header-message relationship which may be difficult to resolve or maintain and suggests that the data formats are ill-conceived.

The following scenario shows a case in which the message's character encoding is changed (via a header targetted at a specific service in the chain of services). The SOAP header prescribing the conversion is correctly removed once the conversion is performed. A variation of this scenario that might represent a problem case would be one where the header remained after the conversion is performed, incorrectly prescribing future conversions. It is easy to imagine scenarios where either the header or the message is modified and the two are no longer properly coordinated.

4.8.1 Character Encoding Conversion Scenario

An environment exists in which a legacy application invokes a service. The receiving service and the legacy requester are not necessarily using the same character encoding scheme. An intermediary site receives the sender's message and, using data in the message's SOAP header, converts the message's character data to the encoding of the ultimate receiver, the service.

A requester sends a SOAP message with an attachment that uses a character encoding and character set of its local process. The ultimate receiver of the message is a service which can only process UTF-8 encoded data. An intermediary provider examines message headers before forwarding the messages to the service's provider. If the character encoding is not UTF-8, the intermediary converts the data, modifies the character encoding attribute of the message, removes the header, and forwards the message.

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope">
  <env:Header actor="...">
    <WS-I18N:convertData
      xmlns:WS-I18N="http://example.org/2002/11/21/WS-I18N"
      env:role="http://www.w3.org/2003/05/soap-envelope/role/next">
        <WS-I18N:senderEncoding>ISO-8859-1</WS-I18N:senderEncoding>
      </WS-I18N:convertData>
    </env:Header>
    <env:Body>
      . . .[legacy ISO-8859-1 data]
    </env:Body>
  </env:Envelope>
```

SOAP message forwarded from intermediary:

```
<?xml version="1.0" encoding="UTF-8" ?>
<env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope">
  <env:Body>
    . . .[UTF-8 data]
  </env:Body>
</env:Envelope>
```

4.9 Service Discovery

Service descriptions are human-readable text intended to describe what the service does and how it should be used. To be useful, the description needs to be a natural language sentence or even a set of keywords in the language that the likely user audience will understand. Searching for Web services depends on language or culture. There should be a way to tag the content with the specific language that it is in and to allow multiple languages. Otherwise false positives or negatives will result.

4.9.1 Searching for Web Services Using UDDI

With respect to internationalization, there are four primary scenarios that will be discussed below:

1. How do I search for services using my language to perform the search?
2. How do I search for services that are specific to my region?
3. How do I search for services that can handle my locale or language preference needs?
4. How do I describe a service that handles multiple locales or languages?

4.9.2 I-026 Searching for Service Descriptions Using My Language

I-026.1 Searching for Service Descriptions using my language

It states in the UDDI Version 3.0.1

[\[UDDI\]](#) specification in the section on *Introduction to Internationalization*:

"1.8.4 Use of Multiple Languages and Multiple Scripts Multinational businesses or businesses involved in international trading at times require the use of possibly several languages or multiple scripts of the same language for describing their business. The UDDI specification supports this requirement through two means, first by specifying the use of XML with its underlying Unicode representation, and second by permitting the use of the `xml:lang` attribute for various items such as names, addresses, and document descriptions to designate the language in which they are expressed. "

Using `xml:lang` and multiple entries, a service provider can publish text information about their service in multiple languages. The name, description, address, and `personName` UDDI elements may have an associated `xml:lang` attribute to indicate the language in which their content is expressed. The `policyDescription` element contains a description of the effect of the policy implementation. This element can also have an `xml:lang` attribute and can appear multiple times to allow for localized versions of the policy description. Providers are encouraged to do this for target language markets that their service may support.

Entity names in UDDI can also provide an Alternate Name in RFC2277 default language, readable in English. This provides a fallback mechanism to allow a search to identify services even if the named contents may be in a script that is not readable by the entity doing the search.

The scenario would be as follows:

1. Service provider publishes service information using UDDI in the provider's default language. The first entity name in a list is considered to be in the primary name and language.
2. Service provider, or other entity, adds localized duplicate content to the UDDI entries for the service.
3. Service requester makes a request for service listings, first setting the primary language for searching using the UDDI Subscription API. The language is indicated by setting the `xml:lang` attribute on query key entities.
4. The UDDI application returns services that match the query in the given `xml:lang` language, matching languages according to the language matching rules defined in [\[RFC3066\]](#).

Here are some examples from the UDDI Version 3.0.1 specification.

Example: Transliteration

Transliteration of the primary name of a business (in this case a Chinese flower shop) in Chinese: its alternative name is a transliteration into Latin script (UDDI calls this a 'romanization'):

```
<businessEntity>
  ...
  <name xml:lang="zh">□□□□</name>
  <name xml:lang="en">Huang He Hwa Dian</name>
  ...
</businessEntity>
```

Example: Translation

Translation in which the primary name of the business is in Chinese, and is a transliteration of its alternative English name. (UDDI calls this 'transliteration')

```

<businessEntity>
  ...
  <name xml:lang="zh">□□□□□□□□□□</name>
  <name xml:lang="en">Compaq Computer Taiwan Limited</name>
  ...
</businessEntity>

```

Example: Multiple Names

The following shows an example of use of multiple name elements to support a multi-script language and also the use of an acronym. In the example, the first `<name>` element is the primary name of the business (a Japanese flower shop) in Japanese Kanji. The second `<name>` element is the business' name transliterated into Japanese Katakana. The third `<name>` element gives the business' full English name, and the fourth `<name>` element gives its English acronym.

```

<businessEntity>
  ...
  <name xml:lang="ja">□□□□□□</name>
  <name xml:lang="ja">□□□□□□□□□□</name>
  <name xml:lang="en">NIPPON FLOWERS </name>
  <name xml:lang="en">NF</name>
  ...
</businessEntity>

```

Where multiple name elements are published, the first name element is treated as the primary name, which is the name by which a business would be searched and sorted in the case of multiply-named businesses. Client applications may use this knowledge to assist in optional rendering of a publisher's primary name or all alternative names.

Developers need to know about the organization and provisioning of multiple names, since the first name element is what UDDI searches and sorts on. Developers must provide a mechanism to override the default behavior if the requester is asking for results in a language other than that of the primary entries based on any `xml:lang` settings in the query structures sent to the UDDI subscription API.

When searching for services, requesters can hunt for names and descriptions by using the UDDI API to pass an optional collection of string values and potentially qualified with `xml:lang` attributes. Since "exactMatch" is the default behavior, the value supplied for the name argument must be an exact match. If the "approximateMatch" findQualifier is used together with an appropriate wildcard character in the name, then any businessService data contained in the specified businessEntity (or across all businesses if the businessKey is omitted or specified as empty) with a matching name value will be returned. Matching occurs using wildcard matching rules. Each name may be marked with an `xml:lang` adornment.

If language markup is specified, the search results report a match only on those entries that match both the name value and language criteria. The match on language is a leftmost case-insensitive comparison of the characters supplied. This allows one to find all services whose name begins with an "A" and are expressed in any dialect of French, for example. Values which can be passed as language criteria are required to obey the rules governing the `xml:lang` attribute (that is, they must be valid language tags or ranges).

UDDI does not specify the use of variant find scenarios to allow alternatives such as accent-insensitive matching. To aid in search retrieval, developers creating a service discovery engine under UDDI may consider alternative match mechanisms.

4.9.3 I-027: Searching for Services Specific to a Region

The UDDI Version 3.0.1 specification states in its introduction to internationalization that UDDI provides features that enable Web service providers to describe the geographical location of different aspects of a business or service, that is, where it offers its products and services, where it is located, or even where it has stores, warehouses, or other branches. This is done through *categoryBags* and *keyedReferences*.

"The optional *categoryBag* element allows *businessEntity* structures to be categorized according to published categorization systems. For example, a *businessEntity* might contain UNSPSC product and service categorizations that describe its product and service offering and ISO 3166 geographical regions that describe the geographical area where these products and services are offered. "

"As within an *identifierBag* , a *keyedReference* contains the three attributes *tModelKey*, *keyName* and *keyValue*. The required *tModelKey* refers to the *tModel* that represents the categorization system, and the required *keyValue* contains the actual categorization within this system. The optional *keyName* can be used to provide a descriptive name of the categorization. Omitted *keyNames* are treated as empty *keyNames*. A *keyName* MUST be provided in a *keyedReference* if its *tModelKey* refers to the general `_keywords` category system. "

"For example, in order to categorize a *businessEntity* as offering goods and services in California, USA, using the corresponding ISO 3166 *tModelKey* within the UDDI Business Registry, one would add the following *keyedReference* to the *businessEntity*'s *categoryBag*: "

```
<keyedReference
  tModelKey="uddi:ubr.uddi.org:categorization:geo3166-2"
  keyName="California, USA"
  keyValue="US-CA" />
```

The use of geographic categorization for services is useful for taxes, import, export, and acknowledgment of available location-specific physical services such as shipping, export, manufacturing, labor, etc.

The problems involved in identifying the actual geographical or political entity that apply to a service or implementation are not dealt with on the level of ISO 3166 country codes or UDDI directory standards and mechanisms for handling geographic location and applicability of services needs further study.

4.10 Introspection of Services When Generating WSDL

Many Web service providers and tools offer the ability to wrap existing code (such as Java Beans or objects, C# methods, or other functions or APIs) with Web services via reflection. Reflection is a process in which software inspects the parameter list and return values of an API call programmatically.

The process of generating WSDL from an object may involve this kind of reflection. Reflection works well with simple, well-understood types such as integers or strings. It works less well with more complex types.

Locale affected types are often a source of problems for reflection. For example, the Java Locale object takes three arguments (language, region, and variant) in its constructor. .NET CultureInfo accepts a slightly modified RFC3066 language identifier. Other locale objects are described by long integers or other values.

Similarly, dates, calendars, and other locale affected data structures may require specific arguments whose semantics and requirements are not available by introspection. The result may be either that the implementer must hide the implementation details using code in the service itself or rely on the requester to provide appropriate data values.

```
<xsd:complexType name="Locale">
  <xsd:all>
    <xsd:element name="language" minOccurs="0" nillable="true" type="xs:string"/>
    <xsd:element name="country" minOccurs="0" nillable="true" type="xs:string"/>
    <xsd:element name="variant" minOccurs="0" nillable="true" type="xs:string"/>
    <xsd:element name="hashCode" minOccurs="0" type="xsd:int"/>
  </xsd:all>
</xsd:complexType>
```

4.11 Ordering, Grouping, and Collation

The ordering or collation of textual data items is a general concern for internationalized software. The problem is exacerbated when the data can be multilingual in nature. For Web services, in scenarios where the ordering of textual data is critical to its correct utilization, it can be difficult to identify the appropriate collation rules to use with sufficient precision to insure those rules are either followed by any services that operate on the data or that appropriate action is taken to compensate for any services that do not use the desired collation rules (for example, by re-sorting the data downstream).

A brief list of these collation issues are described here. An important reference is the Unicode Collation Algorithm (UCA), described by:

[\[UTR10\]](#). Although the UCA is a mature standard, it should be noted that there is wide variance in the implementation of collation algorithms; that few of these implementations are based on UCA; and that there is little or no general agreement on identifiers for collation preferences.

Collation rules cannot be inferred solely from a language identifier or a locale, as the identifiers do not indicate which sort ordering should be used within a locale. A language identifier may be suggestive as to whether a requester expects a particular sort ordering (as with Traditional or Modern ordering in Spanish, for example) but it may not be definitive.

Some examples of sort orderings include: telephone, dictionary, phonetic, binary, stroke-radical or radical-stroke. In the latter two cases, the reference (source standard) for stroke count may also need to be cited.

Different components or subsystems which are used by a software process may employ different sort orderings. For example, a User Agent may provide a drop-down list which sorts the elements of the list at run-time differently from the other components of the agent. Information retrieved from a database may be ordered by an index which has no correlation to the requester's requirements. When different components or subsystems of a Web Service use different collation rules, then errors can occur. They are not always hard errors (i.e. those that generate faults) but the resulting data, operations, or events, may be incorrect or inconsistent with expectations.

In the case of services that might use a binary collation (ordering by the code points of text data) there can be differences in ordering introduced by different components using UTF-8 vs. UTF-16 internally.

Knowing the language of the requester does not prescribe how sensitive the collation should be. Should text elements that are different by case or accent be treated as distinct? Should certain characters be ignored? For example, hyphens are often ignored so that "e-mail" and "email" sort together.

Where case is considered distinct, it may be important to describe whether all lowercase characters precede all uppercase characters, vice versa, or whether they should be intermixed.

Often the performance of an application is impacted by collation. For example, if a service returns results in an unknown ordering, the requester may have to sort the results using its local collation rules. This can consume resources and delay the further use of the results until the entire set can be collated. Alternatively, if results are returned in the order needed by the requester, then the requester can begin to process the first records returned without waiting for the remaining records to arrive.

Of course, collation can be performed at different stages of data processing and timing can be an important consideration. Database indexes are updated as the data is added to the database, not at the time a request arrives. Requests that can use the preordained collation of the index have a significant performance advantage over requests that either cannot use indexes or must re-sort the results.

See [I-009](#) and [I-013](#) for a some examples.

4.12 Natural Language Text Search

Invariably, somewhere along the line, an actual human will use a client application to look for something. When that happens, the search process or processes must perform processing on human readable text. To accommodate this, natural language processing is used. The two primary forms of natural languages processing are language-neutral and language-specific.

4.12.1 Language-Neutral Natural Language Text Search

Most search engines do not understand language, but do understand patterns and proximity. Patterns refer to wildcards and whitespace for full-text search. However, many Asian languages have no concept of whitespace in most full text, and therefore may use a scheme where every character can be considered to be a word.

4.12.1.1 Unicode Normalization

For language neutral applications, text should be normalized to only one form (such as base+combining character or all precomposed) according to Unicode Standard Annex #15

[\[UTR15\]](#) before comparisons are made. For more information, please see [\[CharModNorm\]](#).

Note that this kind of normalization is different from and in addition to other forms of normalization such as case-folding.

4.12.1.2 Catalog or Index in Multiple Languages

Catalogs or indexes, if more than one language is to be supported should contain language variants of keywords. This can be populated automatically, with translations done semi-automatically, using context to aid in creating the right alternate terms.

This is the ability to have one catalog or index item, with the description in many languages. The service wants to be able to update price and quantity in one place per item and have that reflected across all languages. The client wants to search for items in their own language.

In the following business XML, a catalog item is defined as follows:

```
<elementtype name="Product">
  <model>
    <sequence>
```



```

    ...
    <element type="<ShortDescription.html>" occurs="*" />
    <element type="<LongDescription.html>" occurs="*" />
    ...

```

The Descriptions can occur from zero to many times.

The description might be defined as follows:

```

<elementtype name="ShortDescription">
  <model>
    <string/>
  </model>
  <attdef name="lang" datatype="xml:lang" prefix="xml">
    <default>en</default>
  </attdef>
</elementtype>

```

You should then be able to support the following:

```

<Product Type="Good" SchemaCategoryRef="C43171803">
  <ProductID>154723-005</ProductID>
  <Manufacturer PartnerRef="Acme Tools"></Manufacturer>

  . . .
  <ShortDescription xml:lang="en">Wrench</ShortDescription>
  <ShortDescription xml:lang="en_GB">Spanner</ShortDescription>
  <ShortDescription xml:lang="da">fladnoegle</ShortDescription>
  <ShortDescription xml:lang="es-ES">llave abierta</ShortDescription>
  <ShortDescription xml:lang="es-MX">llave inglesa</ShortDescription>
  <ShortDescription xml:lang="fr-FR">clef à fourche</ShortDescription>
  <ShortDescription xml:lang="de">Gabelschlüssel</ShortDescription>
  <ShortDescription xml:lang="it">chiave a forchetta</ShortDescription>
  <ShortDescription xml:lang="ja">□□□</ShortDescription>
  <ShortDescription xml:lang="ko">□□□</ShortDescription>
  <ShortDescription xml:lang="nl">vorkvormige sleutel</ShortDescription>
  <ShortDescription xml:lang="pt-PT">chave fixa</ShortDescription>
  <ShortDescription xml:lang="pt-BR">chave de boca</ShortDescription>
  <ShortDescription xml:lang="zh-CN">□□</ShortDescription>
  <ShortDescription xml:lang="zh-TW">□□</ShortDescription>
  . . .
</Product>

```

4.12.2 Language-Specific Natural Language Text Search

Most search engines that have any linguistic characteristics are tuned to a specific language such as English, German, or French. This allows techniques, such as stemming and ignoring stop-words to operate according to the unique characteristics of the language it is operating in.

4.12.2.1 Keyword Searching

As with language-neutral processing, most search engines don't actually understand the language, but do understand patterns and proximity. Patterns refer to wildcards and whitespace for full-text search. However, many Asian languages have no concept of whitespace in full text, and therefore may use a scheme where every character can be considered to be a word.

When searching for keywords, language must be considered to resolve some items such as abbreviations. For example, in the string, "422 St. Jerome St.", "St." could be either "Saint" or "Street".

4.12.2.2 Gender and plural variants

Some terms have more than one form depending on the gender or plurality of the object. As an example, "Dr. Alvarez" or "Doctor Alvarez" in English, could be either "Dr" or "Dra" for "Doctor" or "Doctora" in Spanish. Therefore, to increase the number of valid hits in the absence of context, a service should match all variants of a matching term if translated to an alternate language.

4.12.2.3 Orthographic Variation in Searching ('like' clauses)

When operating in a specific language, further normalization may be required in addition to abbreviation expansion and character normalization. This is to accommodate variant spellings for the same word. In German, for instance, "Müller" should also return "Mueller" to allow searching across legacy and alternate systems.

4.12.2.4 Use of Intermediary Translation and Dictionary Look-Up Service

To allow a service to provide search services from clients in other languages, the service could do the search more than once, depending on implementation design. First in the original text as submitted by the client, and in second or subsequent searches after submitting the original query to a translation or dictionary look-up service.

As an example, the address "422 St. Jerome St." could be also be represented as:

en:	422 Saint Jerome Street
fr:	Rue De 422 Saints Jerome
es:	Calle De 422 Santos Jerome
de:	422 Heiliger Jerome Straße
ja:	□Jerome□422□□

The query would look something like this:

```
Client ==> <query xml:lang="lang0"> ==> service ==> <query xml:lang="lang
^== look-up service
v
==> <query xml:lang="lang1">
```

```
==> <query xml:lang="lang2">  
:  
==> <query xml:lang="langN">
```

4.12.2.5 Phonetic Searches

Note that phonetic searches, such as "Soundex" are usually tuned to specific language characteristics. Soundex, for example, was designed for the U.S. Census Bureau in 1890, and first patented in 1918 to allow phonetic sorting of English surnames. It has poor precision, is unable to handle multicultural names, produces many false positives and misses many potentially correct terms. That being said, there exist proprietary phonological name matching software that produces better results across languages and cultures, but it must be tested and implemented with the caveat that phonetic searching across languages is inherently fraught with errors due to the dialectical differences.

4.13 Locale Sensitive Presentation and Human Readable Text

4.13.1 I-021: Data Formatting for End User on Receiver Side

Data is formatted for an end user by the receiver according to the end user's preferences and the system conventions.

The receiver may format data in order to display the data in a user interface. Locale sensitive data formatting functions are widely provided by internationalization functionality of operating systems, programming languages, or applications such as word processors and middleware. Therefore, an application may be able to format most locale neutral data using built-in internationalization functions. The details of data formatting vary across different systems and there may be dependencies on the data provided or on third-party data sources. Therefore, Web services do not guarantee identical presentation of the same data on different systems.

4.13.2 I-022: Data Formatting on Sender Side

Many applications produce human readable text as output. This makes it necessary for the service to format data for viewing by an end user on the receiver side. The service must choose which language message to return and the format of data inserted into the message. The language and locale preferences applied may or may not be the same and these variations could result in human readable messages that are perceived as flawed by the end user.

Example: Populating a News Feed

A service is created to populate an RSS news feed with stories from a database. Each of the following examples might be returned for same story:

A. "Theatre Center News: The date of the last version of this document was 2003 3 20. A copy can be obtained for \$5.000,0 or 123.400,57 py6. We would like to acknowledge contributions by the following authors (in alphabetical order): Alaa Ghoneim, Behdad Esfahbod, Ahmed Talaat, Eric Mader, Asmus Freytag, Avery Bishop, and Doug Felt."

B. "Theater Center News: The date of the last version of this document was 3/20/2003. A copy can be obtained for \$5,000.00 or 123,400.57 Russian rubles. We would like to acknowledge contributions by the following authors (in alphabetical order): Alaa Ghoneim, Ahmed Talaat, Asmus Freytag, Avery Bishop, Behdad Esfahbod, Doug Felt, Eric Mader."

C. "Theatre Centre News: The date of the last version of this document was 20/3/2003. A copy can be obtained for \$5,000.00 or 123,400.57 Russian rubles. We would like to acknowledge contributions by the following authors (in alphabetical order): Alaa Ghoneim, Ahmed Talaat, Asmus Freytag, Avery Bishop, Behdad Esfahbod, Doug Felt, Eric Mader."

Clearly there are many possible (indeed acceptable) variations of the example texts. For example, one might quibble about the use of first vs. last name sorting in the list, but clearly the first list was not acceptable English alphabetical order. If one is quoting a name, like "Theatre Centre News", it might be left in the source orthography even if it differs from the publication target orthography. And so on. However, just as clearly, there limits on what is acceptable English (in this case), and 2003 3 20, for example, is not.

4.13.3 Enumerated Values and Object Names

Occasionally, in the design of an application, there is a need to present enumerated values to an end-user to choose an option for completion of a transaction. Take, for example, an on-line purchase for travel. The credit cards, "MasterCard®" and "Visa®" have European counterparts known as "Eurocard®" and "Carte Bleu®" respectively. Each of these go through the same payment clearinghouses and are processed in the same way. It is common to see European payment sites stating that Mastercard/Eurocard and Visa/CarteBleu are honored. However, outside of Europe, only the more common global brands are used.

In this example, a requester includes a user profile in a request for valid payment methods to present to the user. The profile includes the language and country of residence of the user. The service then returns an enumerated list of accepted payment methods. The requester may have no previous knowledge of the payment methods, and so will rely on the service to provide the appropriate types to present to the user. The service will expect one of the values in the enumerated list to be returned as the payment type when completing the transaction.

From the Open Travel Alliance (OTA), we find the following valid enumeration values for credit cards:

(From
<http://www.opentravel.org/2004A/xsd/3/simpletype/PaymentCardCodeType.htm>)

EnumerationDescription

- AX American Express
- BC Bank Card
- BL Carte Bleu
- CB Carte Blanche
- DN Diners Club
- DS Discover Card
- EC Eurocard
- JC JCB Credit Card
- MC MasterCard
- TP Universal Air Travel Card
- VI Visa

Each of these would normally be presented in a pull-down pick list, but a particular organization may wish to discard with the differences, and present the list in Europe as:

```
...  
VI Visa/Carte Bleu  
MC Eurocard/MasterCard  
...
```

and in North America as:

```
...  
MC MasterCard  
VI Visa  
...
```

4.13.3.1 Use of Default English-like Names

In an enumerated list, best practices dictate that there is an internal value from which actions are determined, and an external value, which is presented to an end user. In the same way, even if a service has been told what locale a requester is

using, the service may want to provide alternate default display values that can be used for debugging purposes or bilingual status or transaction displays.

4.13.3.2 Types of Names

Names in an enumerated list fall into three primary categories, each used for different purpose.

Display names are those which are presented to the user. A well designed system will have only the Requester generating and displaying these values. If necessary, a Requester should be able to request a set of standard localized names from a Service. Those names should be keyed to culture-neutral values that are used for communication between processes.

(from <http://www.opentravel.org/2004A/xsd/9/complextype/SeatingPrefType.htm>)

Since credit card names are trademarked and don't change much from one language to another, the example below will use seat selection for "Aisle", "Center", "Window" as an example.

The OTA Schema, for instance, passes strings for this value, but an end-user will want to see the values in their own language. In German, the choices would be Gang, Mitte, and Fenster. Therefore, the designer of the Service may wish to return qualified locale sensitive display names that the Requester can use for display to the end user. This is especially useful for instances where terminology or messages might change and it is not desired to update the whole requester application just to update a few terms.

```
<xs:attribute name="SeatPosition" type="xs:string" use="optional">
  <xs:annotation>
    <xs:documentation xml:lang="en">Preferred position of seat in a
      such as Aisle, Center, Window, etc.</xs:documentation>
  </xs:annotation>
  <xs:simpleType>
    <xs:restriction base="StringLength16">
      <xs:enumeration value="Aisle">
        <displayName xml:lang="en">Aisle</display>
        <displayName xml:lang="de">Gang</display>
      </xs:enumeration>
      <xs:enumeration value="Center">
        <displayName xml:lang="en">Center</display>
        <displayName xml:lang="de">Mitte</display>
      </xs:enumeration>
      <xs:enumeration value="Window">
        <displayName xml:lang="en">Window</display>
        <displayName xml:lang="de">Fenster</display>
      </xs:enumeration>
    </xs:restriction>
  </xs:simpleType>
</xs:attribute>
```

The actual enumeration value, not the displayName, would be used internally and passed from Requester to Service, while the displayName would be presented to the user interface according to locale.

4.14 Data Structures

4.14.1 Times and Time Zones

Date and time handling in Web services is affected by time zones. However, there is no standard parameter to indicate the time zone alone. Locales are not useful for determining time zone because there can be many time zones within a given locale. Applications' needs with regard to time zone also differ.

For example, for a single timestamp the RFC 3339 UTC offset is sufficient. For processing dates (such as calculating a duration) you may need more information (such as daylight savings or summer time rules, if they apply).

Scenario A: A Web service returns the current time of a city listed as part of the request. The requester sends the name of a city (with an `xml:lang` attribute value) and the provider returns the current time in that city formatted in

[\[ISO8601\]](#) format (hh:mm:ss).

Scenario B: A Web service takes a date/time value in ISO 8601 format (yyyymmddThhmm+hhmm) and the name of a city with an `xml:lang` attribute value, and returns the value converted to the specified city's time zone.

Scenario C: As a sub-process of a "meeting manager" service, a Web service inspects multiple appointment books looking for mutually available time slots. The requester provides a span of time in ISO 8601 format (yyyymmddThhmm+hhmm) using a start time and an end time. The inspected appointment books store information about their time zones. The service returns a series of time spans in the ISO 8601 format.

4.14.2 Calendars and Dates

A Web service is set up to calculate a calendar date and send it back to the requester. The value returned represents a specific date on the calendar, not a timestamp value as might be associated with a particular locale or time zone. The service may need to take in information such as the calendar type, year, and related descriptive information.

Scenario A: A service calculates the date for Easter, Passover, or Ramadan for any given year, returning a date value in ISO 8601 format. These religious holidays are partly based on astronomical events, such as lunar phases, as well as historical tables. They are not strictly calendar dependent in the way that many secular holidays, such as various national independence days or leader's birthdays are, nor are they predictable, for example, the fourth Thursday in November. Thus the need for a service to calculate the date might be necessary. The SOAP request would contain a holiday and a year in ISO 8601 format. In

addition, some other data may be required, such as for Easter there may be a parameter specifying "Orthodox" or "Western". The Web service would in turn calculate the appropriate date and send a message back to the requester with the calculated date. Some other service may be used to convert the returned date value into a specified calendar type, such as the Japanese calendar.

Scenario B: A service calculates historical dates in different parts of the world and returns an equivalent ISO 8601 date to the requester. The SOAP request would contain a date and its country of origin. For example, a request might have the date 1812-08-26 and the origin "Russia". Russia was using a different calendar from places such as Italy or France at that time; what would appear as the same date was actually several days different. While this may look like it is part of the locale due to the country of origin, it should not be treated as such. Locales are typically associated with the end user, not with a piece of data, and a locale does not contain information on historical times.

Scenario C: A service calculates Chinese New Year for any non-Chinese calendar type. The SOAP request would include a parameter with the calendar type, such as "Gregorian", "Hebrew", or "Japanese Imperial". The locale is irrelevant to the calculation, since the requester may be looking for information unrelated to user preferences or system settings.

4.15 Legal and Regulatory Goobers

4.15.1 Modeling Tax, Customs, Legal, and Other Cross-Border and Cultural Considerations

Tax, customs, legal, and similar matters are usually country-specific. However, much of the types of processing involved are the same. For example, many tax calculations take a percentage or set of percentages of a given amount. A set of Web services can work together to provide information for many countries, avoiding code and process duplication.

There is more information needed in these types of processes than just the country identifier. Language information is crucial for legal documents, and important for other regime-type operations as well. For tax calculations, the currency of the incoming values as well as the currency of the result must be specified. Other cross-border services will likely require other types of information, such as address formats or some sort of legal status indicator.

Scenario A: Service A, specific to Country C, takes in the value of a sale, a language parameter, and the names of the city and the province. The currency is limited to Country C's official currency. Service A then calls a set of services, translating names into identifiers. Service B takes in a city id and a monetary value, then calculates city sales tax based on current tax tables it retrieves from other services; it returns the tax amount as a numeric value. Service C performs a similar function for taxes at the provincial level.

Service A then takes those monetary values and returns them with identifying tags for the city and provincial tax.

If Service A were to be used for multiple countries, there would have to be additional parameters, for example:

- a country identifier
- other regional identifiers, such as county, state, province, or region
- a currency identifier

There would have to be a function to handle currency calculations, possibly in a separate service. The additional tax regions need to be managed, again by separate services.

Scenario B: An application uses a Web service to send DVDs to rental customers around the world. DVDs contain a region code that limits where they can be played (according to the country they are intended for.) The Web service takes the country ID of the customer and selects the right region code DVD to send.

Scenario C: A service takes a country ID, looks it up in a database, and return the driving rules for that country.

Configuration or business logic may have to be carefully designed in order to deal with these kinds of issues.

4.16 Transports

Web services may use a variety of transport technologies and protocols. Many of these have parameters defined for data identification. Some of these parameters are necessary for proper processing of international data. The specifics of several transport protocols are discussed in this section.

Note that each transport has different capabilities regarding internationalization-related metadata and that the transport stack that processes this metadata and the service provider may not be the same process. Although transports can sometimes pass interesting data about the requester, this doesn't fit well with the general Web services design of using XML data structures in a composable way: the metadata can get lost too easily and doesn't reflect the needs of the service provider accurately.

4.16.1 HTTP

The HyperText Transport Protocol (HTTP) is often used for Web service message transport. HTTP contains some header fields which are useful for identifying sender preferences and capabilities. One of these fields of interest when internationalizing a service is Accept-Language.

Accept-Language takes one or more language identifiers in RFC3066 (or its successor) format as its parameters. Each language identifier can have a quality value which gives a relative priority. Here is an example:

```
Accept-Language: zh-cn, fr-ch;q=0.8, fr;q=0.7
```

The above could be read as "Simplified Chinese is preferred, but Swiss French is acceptable, as are other types of French." There is more information about the handling of Accept-Language in the HTTP 1.1 specification.

A Web service requester using HTTP can include an Accept-Language field to indicate the languages preferred. The provider could then take that information and use it to return human-readable data in the appropriate language.

Some Web application environments, such as .NET or J2EE, use this field to try and determine the user locale for use in application logic. As noted above, while this might be an acceptable short term strategy for some implementations, the design doesn't fit with the distributed nature of Web services. At best, a Web service provider can merely infer some information about the requester or the requester's host from this field and this may not be sufficient to provide a high quality of service.

4.16.2 FTP

File Transfer Protocol (FTP) is a simple transport mechanism that can be used to transport Web service documents. The main international consideration in using FTP is to specify the representation type as I (Image), allowing 8-bit values to pass unchanged through the transfer.

File names, path names, and character encoding issues may intrude here, since the FTP specification provides limited guidance.

4.16.3 SMTP

Simple Mail Transfer Protocol (SMTP) has no particular provisions for international data. SMTP itself is limited to 7-bit data, but can transport 8-bit data. Its main restriction is an 8-bit gateway; that is, encodings such as UTF-16 and UTF-32 may not be successfully transmitted and should be avoided.

4.16.3.1 MIME Tags

Multipurpose Internet Mail Extensions (MIME) tags are necessary for a multipart SOAP request, for example, when transmitting the SOAP message itself or a SOAP attachment. MIME contains a number of headers which may be used for international data.

MIME can identify the character encoding of attachments (via the Content-Type charset attribute). This is useful when the attachment does not identify its own charset internally. Examples of such attachments are plain text documents which cannot contain markup identifying the character encoding and legacy markup documents which do not contain a charset tag by omission. If the attachment contains an internal charset tag, the MIME charset parameter should be omitted to avoid an inadvertent mismatch.

MIME can also contain a Content-Language tag. While it is better to indicate the document language inside the document itself, sometimes it isn't possible. For example, if there is an image attachment which contains embedded text, the Content-Language header can identify the language externally.

Interpreting MIME Content-Language headers as a locale is problematic. Although the HTTP transport's Accept-Language header is sometimes use for this purpose, Content-Language generally indicates the actual language of the content.

See the example in

[4.5.2 Character Encoding of Attachments.](#)

4.16.4 IRIs, URIs, and fun stuff

Internationalized Resource Identifiers (IRIs, see

[\[IRI\]](#)) should be used wherever URIs would be used, to allow the use of non-ASCII characters in a natural way. This can be done automatically by using the anyURI data type from XML Schema [\[XMLS-2\]](#).

4.17 Orchestration and Choreography

Orchestration of Web services involves the creation of a sequence of services, generally wrapped as a single process or thread of execution.

Choreography is a higher-level form of orchestration, generally involving orchestration of Web services that span multiple servers or systems.

Orchestration and choreography include many of the attributes of traditional business processes, such as transactionality, security, reliability, and so forth. They may involve the interaction between Web services, applications, and human interactive workflow processes.

In many cases orchestrated or choreographed sets of services may be wrapped so that they appear as a single service that itself can be invoked.

The locale patterns, policies, and their interactions previously discussed in this document apply to the results of orchestrated collections of Web services, as well as to the individual services that make up the orchestrated service. In fact, the locale patterns may form part of the business logic or influence the execution, selection of service targets, and other operational or runtime attributes of an orchestration.

A References (Non-Normative)

[CHARMOD]

"Character Model for the World Wide Web 1.0: Fundamentals", Martin J. Dürst, François Yergeau, Richard Ishida, Misha Wolf, Tex Texin, W3C

Working Draft 25 February 2004. (See <http://www.w3.org/TR/2004/WD-charmod-20040225/>.)

[CharModNorm]

"Character Model for the World Wide Web 1.0: Normalization", Martin Dürst, François Yergeau, Richard Ishida, Misha Wolf, Tex Texin, Addison Phillips, W3C Working Draft 25 February 2004. (See <http://www.w3.org/TR/2004/WD-charmod-norm-20040225/>.)

[IRI]

"Internationalized Resource Identifiers (IRIs)", Martin Dürst and Michel Suignard, Internet Draft draft-duerst-iri-09.txt (work in progress, see <http://www.w3.org/International/iri-edit/>), 19 July 2004. (See <http://www.w3.org/International/iri-edit/draft-duerst-iri-08.txt>.)

[ISO8601]

"Representations of dates and times", ISO (International Organization for Standardization), 1988-06-15. (See <http://www.iso.org/iso/en/CatalogueDetailPage.CatalogueDetail?CSNUMBER=26780&ICS1=1&ICS2=140&ICS3=30>.)

[JITXL]

"JIT Localization", Mark Davis, IBM, 21 March 2003 (See http://oss.software.ibm.com/cvs/icu/~checkout/~icuhtml/design/jit_localization.ht

[RFC822]

"Standard for the Format of ARPA Internet Messages", Revised by David H Crocker, 13 August 1982. (See <http://www.ietf.org/rfc/rfc822.txt>.)

[RFC2277]

IETF Policy on Character Sets and Languages, Harald Alvestrand, RFC 2277, January 1998. (See <http://www.ietf.org/rfc/rfc2277.txt>.)

[RFC2616]

"Hypertext Transfer Protocol -- HTTP/1.1", R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, T. Berners-Lee, RFC 2616, June 1999. (See <http://www.ietf.org/rfc/rfc2616.txt>.)

[RFC2781]

UTF-16, an encoding of ISO 10646, P. Hoffman, F. Yergeau, RFC 2781, February 2000. (See <http://www.ietf.org/rfc/rfc2781.txt>.)

[RFC3066]

Tags for the Identification of Languages, H. Alvestrand, RFC 3066, January 2001. (See <http://www.ietf.org/rfc/rfc3066.txt>.)

[RFC3629]

UTF-8, a transformation format of ISO 10646, F. Yergeau, RFC 3629, STD 63, November 2003. (See <http://www.ietf.org/rfc/rfc3629.txt>.)

[SOAP-0]

"SOAP Version 1.2 Part 0: Primer", Nilo Mitra, W3C Recommendation 24 June 2003. (See <http://www.w3.org/TR/2003/REC-soap12-part0-20030624/>.)

[SOAP-1]

"SOAP Version 1.2 Part 1: Messaging Framework", Martin Gudgin, Marc Hadley, Noah Mendelsohn, Jean-Jacques Moreau, Henrik Frystyk Nielsen, W3C Recommendation 24 June 2003. (See <http://www.w3.org/TR/2003/REC-soap12-part1-20030624/>.)

[SOAP-2]

"SOAP Version 1.2 Part 2: Adjuncts", Martin Gudgin, Marc Hadley, Noah Mendelsohn, Jean-Jacques Moreau, Henrik Frystyk Nielsen, W3C

Recommendation 24 June 2003. (See <http://www.w3.org/TR/2003/REC-soap12-part2-20030624/>.)

[SOAP-AF]

"SOAP 1.2 Attachment Feature", Henrik Frystyk Nielsen, Hervé Ruellan, W3C Working Group Note 8 June 2004. (See <http://www.w3.org/TR/2004/NOTE-soap12-af-20040608/>.)

[SOAP-EB]

"SOAP Version 1.2 Email Binding", Highland Mary Mountain, Jacek Kopecky, Stuart Williams, Glen Daniels, Noah Mendelsohn, W3C Note 3 July 2002. (See <http://www.w3.org/TR/2002/NOTE-soap12-email-20020703/>.)

[UDDI]

"UDDI Version 3.0 Published Specification", Tom Bellwood, Luc Clément, David Ehnebuske, Andrew Hately, Maryann Hondo, Yin Leng Husband, Karsten Januszewski, Sam Lee, Barbara McKee, Joel Munter, Claus von Riegen, 19 July 2002. (See http://uddi.org/pubs/uddi_v3.htm.)

[UTR10]

"Unicode Technical Standard #10: The Unicode Collation Algorithm", Mark Davis, Kenneth Whistler, Version 4.0, Updated 08 January 2004. (See <http://www.unicode.org/reports/tr10/>.)

[UTR15]

"Unicode Technical Standard #15: Unicode Normalization Forms", Mark Davis, Martin Dürst, Version 4.0.0, Updated 17 April 2003. (See <http://www.unicode.org/reports/tr15/>.)

[WSA]

"Web Services Architecture", David Booth, Hugo Haas, Francis McCabe, Eric Newcomer, Michael Champion, Chris Ferris, David Orchard, W3C Working Group Note 11 February 2004. (See <http://www.w3.org/TR/2004/NOTE-ws-arch-20040211/>.)

[WSAG]

"Web Services Glossary", Hugo Haas, Allen Brown, W3C Working Group Note 11 February 2004. (See <http://www.w3.org/TR/2004/NOTE-ws-gloss-20040211/>.)

[WSAR]

"Web Services Architecture Requirements", Daniel Austin, Abbie Barbir, Christopher Ferris, Sharad Garg, W3C Working Group Note 11 February 2004. (See <http://www.w3.org/TR/2004/NOTE-wsa-reqs-20040211/>.)

[WSAUS]

"Web Services Architecture Usage Scenarios", Hao He, Hugo Haas, David Orchard, W3C Working Group Note 11 February 2004. (See <http://www.w3.org/TR/2004/NOTE-ws-arch-scenarios-20040211/>.)

[WSDL-B]

"Web Services Description Language (WSDL) Version 1.2 Part 3: Bindings", Jean-Jacques Moreau, Jeffrey Schlimmer, W3C Working Draft 11 June 2003. (See <http://www.w3.org/TR/2003/WD-wsdl12-bindings-20030611/>.)

[WSDL-V20]

"Web Services Description Language (WSDL) Version 1.2", Roberto Chinnici, Martin Gudgin, Jean-Jacques Moreau, Jeffrey Schlimmer, Sanjiva Weerawarana, W3C Working Draft 26 March 2004. (See <http://www.w3.org/TR/2004/WD-wsdl20-20040326/>.)

[WSRequirements]

"Requirements for the Internationalization of Web Services", Addison Phillips, W3C Working Draft 17 December 2003. (See <http://www.w3.org/TR/ws-i18n-req/>.)

[XFORMS]

"XForms 1.0", Micah Dubinko, Leigh L. Klotz, Jr., Roland Merrick, T. V. Raman, W3C Recommendation 14 October 2003. (See <http://www.w3.org/TR/2003/REC-xforms-20031014/>.)

[XHTML]

"XHTML 1.0 The Extensible HyperText Markup Language (Second Edition)", W3C Recommendation 26 January 2000, revised 1 August 2002. (See <http://www.w3.org/TR/xhtml1/>.)

[XML]

"Extensible Markup Language (XML) 1.0 (Third Edition)", Tim Bray, Jean Paoli, C. M. Sperberg-McQueen, Eve Maler, François Yergeau, W3C Recommendation 04 February 2004. (See <http://www.w3.org/TR/2000/REC-xml-20001006/>.)

[XML-JP]

"XML Japanese Profile", MURATA Makoto, W3C Note 14 April 2000. (See <http://www.w3.org/TR/2000/NOTE-japanese-xml-20000414/>.)

[XMLS-2]

"XML Schema Part 2: Datatypes", Paul V. Biron, Ashok Malhotra, W3C Recommendation 02 May 2001. (See <http://www.w3.org/TR/2001/REC-xmlschema-2-20010502/>.)

B Acknowledgements (Non-Normative)

This document is the work of the Web Services Task Force of the W3C Internationalization Working Group.

The example in

[I-022](#) was taken directly from an example by Mark Davis, IBM, and is used by his permission.

C Heisei (Non-Normative)

The imperial Calendar is commonly used in Japan. Heisei is the current era, and it started on January 8th, 1989. Year 2002 is Year Heisei 14. Showa is the previous era, which ended on January 7th, 1989 or Showa 64.